

Vyhledávání

Karel Richta a kol.

Přednášky byly připraveny s pomocí materiálů, které vyrobili Marko Berezovský, Petr Felkel, Josef Kolář a Michal Píše

Katedra technických studií
Vysoká škola polytechnická v Jihlavě

© Karel Richta, 2019

Základy strukturovaného programování, ZSP 11/2019, Lekce 10

<https://moodle.vspj.cz/course/view.php?id=200356>



Vysoká škola
polytechnická
Jihlava



Vyhledávání (Searching)

Definice:

- Vstup: množina n záznamů obsahujících klíče a hledaný klíč k
- Výstup: Záznamy s klíčem k nebo informace, že žádný takový neexistuje

Implementace:

- Implementace pomocí pole:
 - Sekvenční vyhledávání
 - Binární vyhledávání
- Binární vyhledávací stromy (BVS, nebo BST – Binary Search Trees):
 - Reprezentace uzlů a hran
 - Operace nad BVS
 - Vyvažování BVS

Typické operace při vyhledávání

- Připomeňme si operace související s vyhledáváním:
 - vložení záznamu s klíčem
 - odstranění záznamu s klíčem
 - vyhledání záznamu podle klíče
 - minimum a maximum
 - předchůdce a následník
 - a další ...

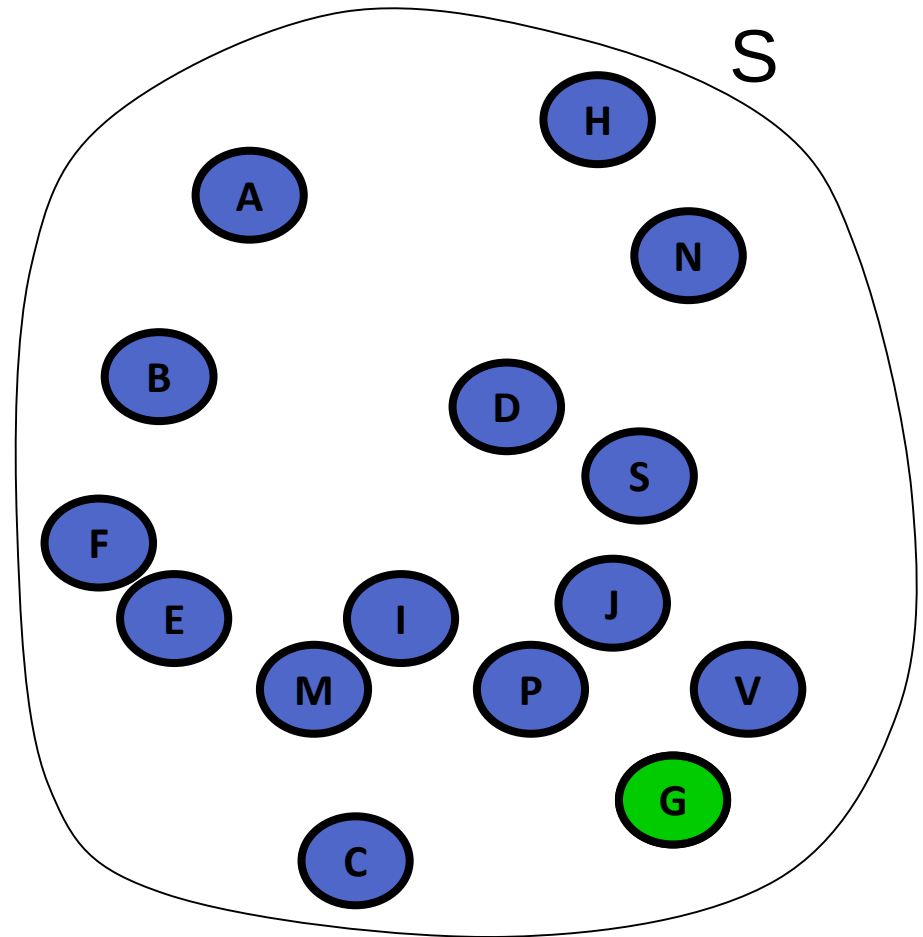
Vyhledávání

Vstup: množina n klíčů a hledaný klíč k

Popis problému: Kde je k ?

G?

- Hledání G v S bylo úspěšné



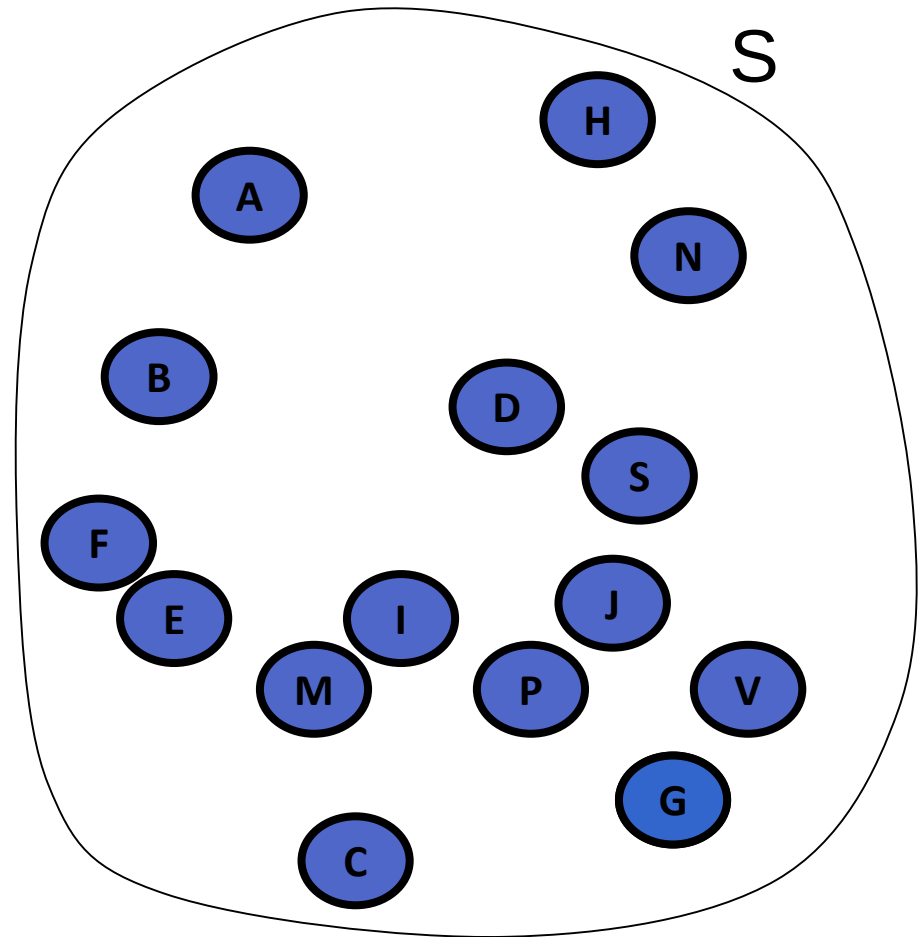
Vyhledávání

Vstup: množina n klíčů a hledaný klíč k

Popis problému: Kde je k ?

L?

- Hledání L v S nebylo úspěšné



Prostor pro vyhledávání

Prohledávaný prostor S (search space)

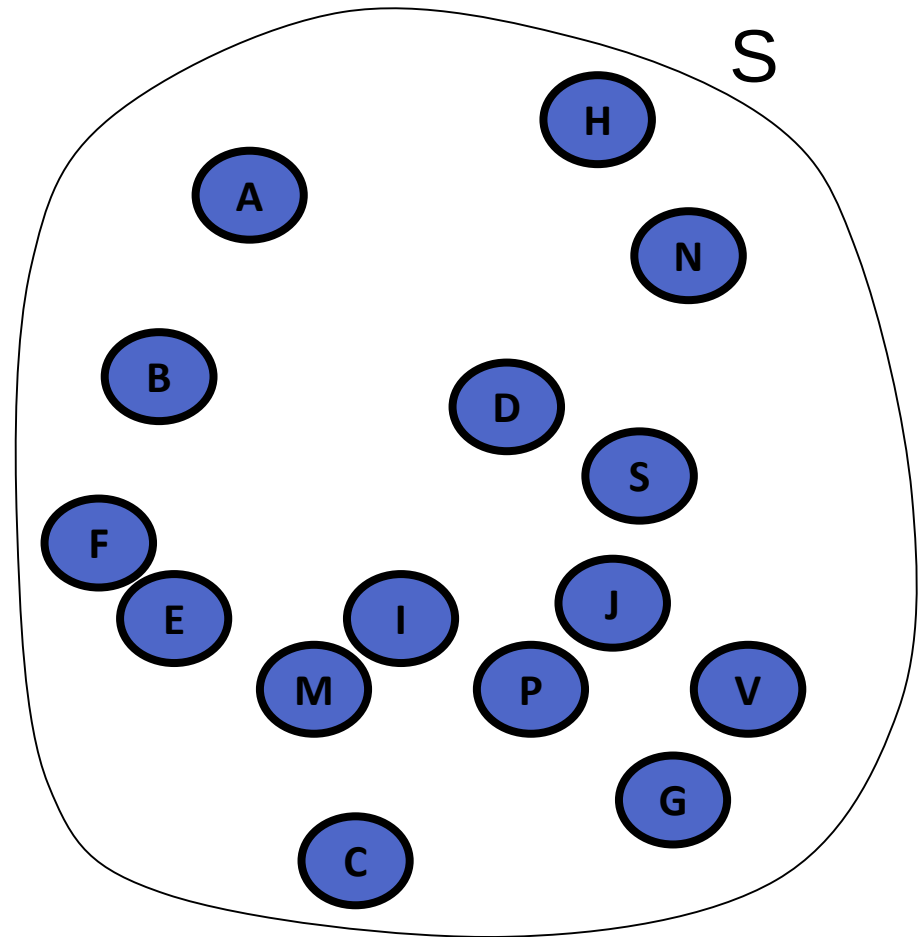
= množina klíčů, ve které hledáme

- přesněji: množina záznamů, ve které hledáme podle klíče
- unikátní klíče
- tabulka, soubor, ...

Universum U pro vyhledávací prostor

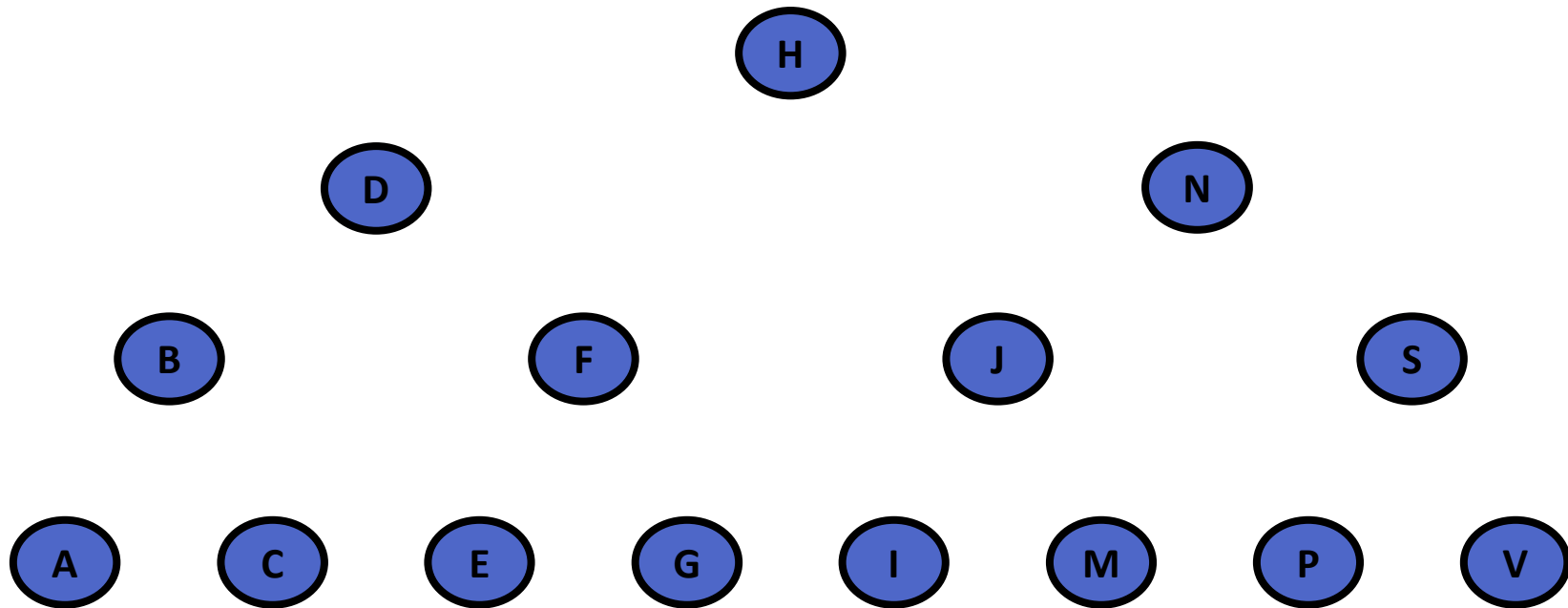
= množina všech možných klíčů

$$S \subset U$$



Problémy vyhledávání

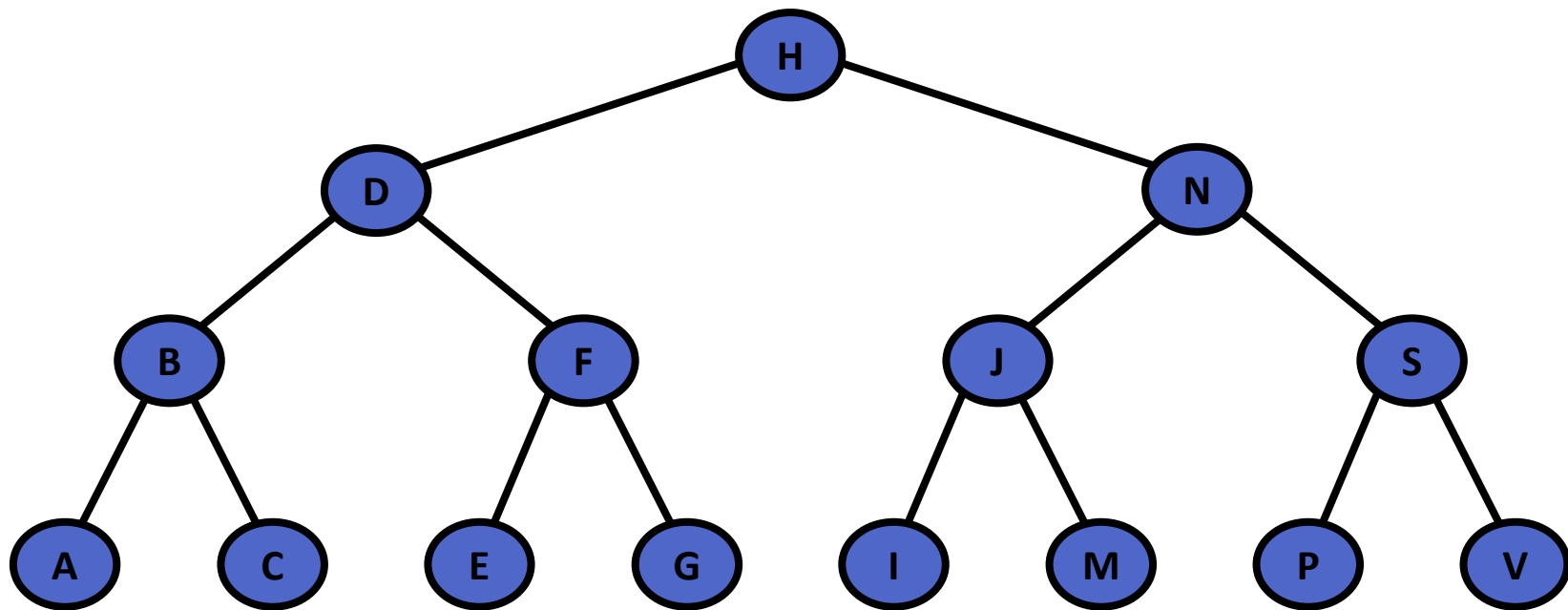
- Rychlost



Problémy vyhledávání

Vstup: množina n klíčů a hledaný klíč k

Popis problému: Kde je k ?

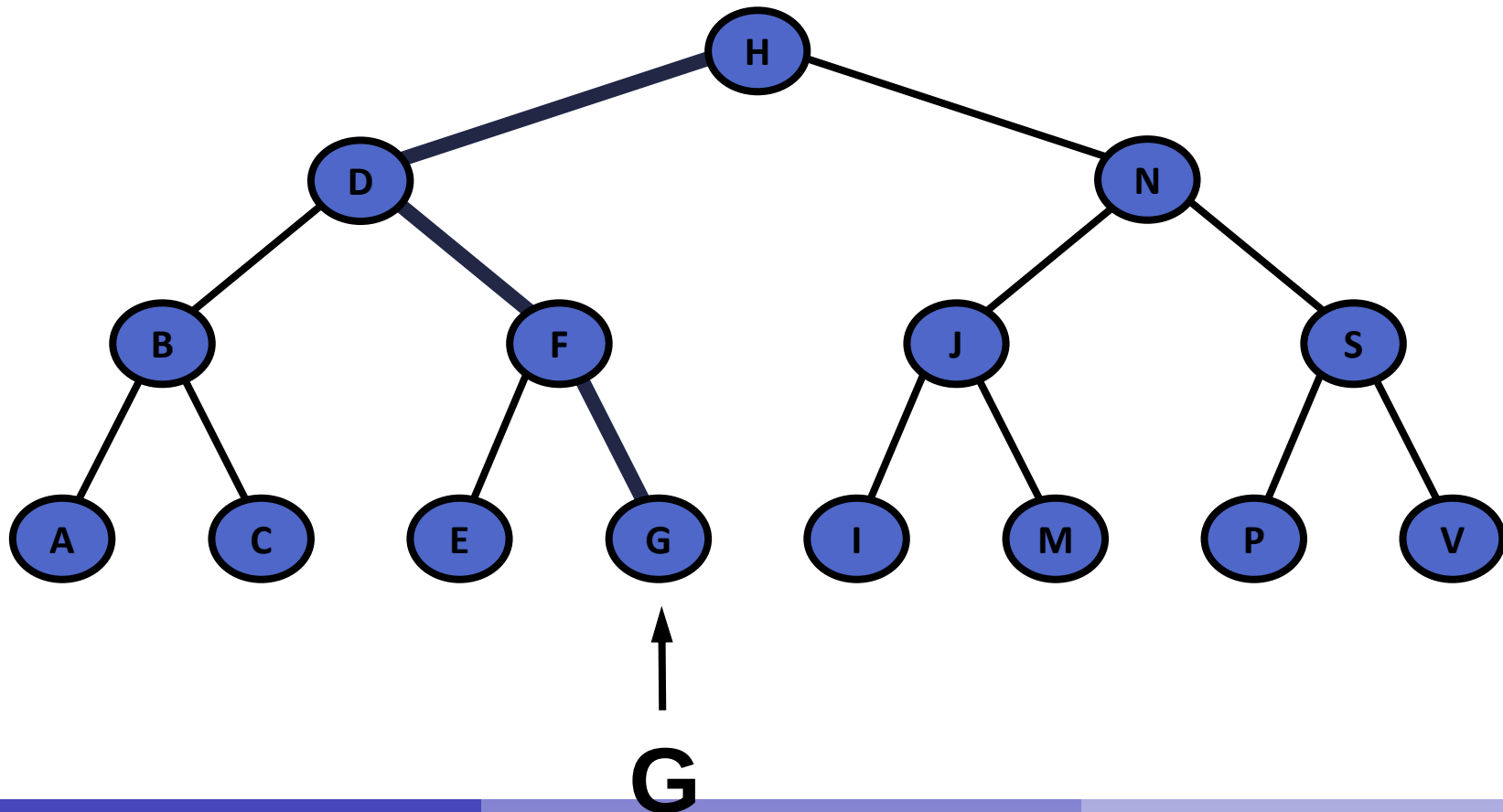


G?

Problémy vyhledávání

Vstup: množina n klíčů a hledaný klíč k

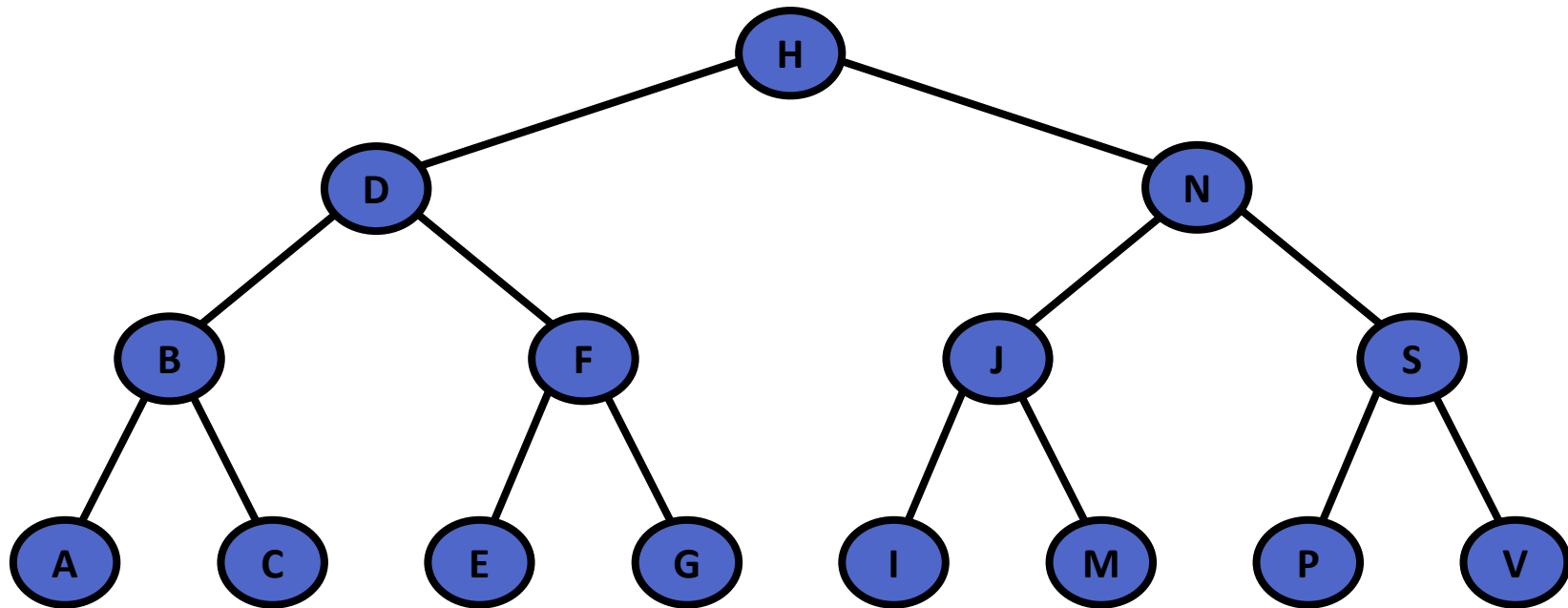
Popis problému: Kde je k ?



Problémy vyhledávání

Vstup: množina n klíčů a hledaný klíč k

Popis problému: Kde je k ?

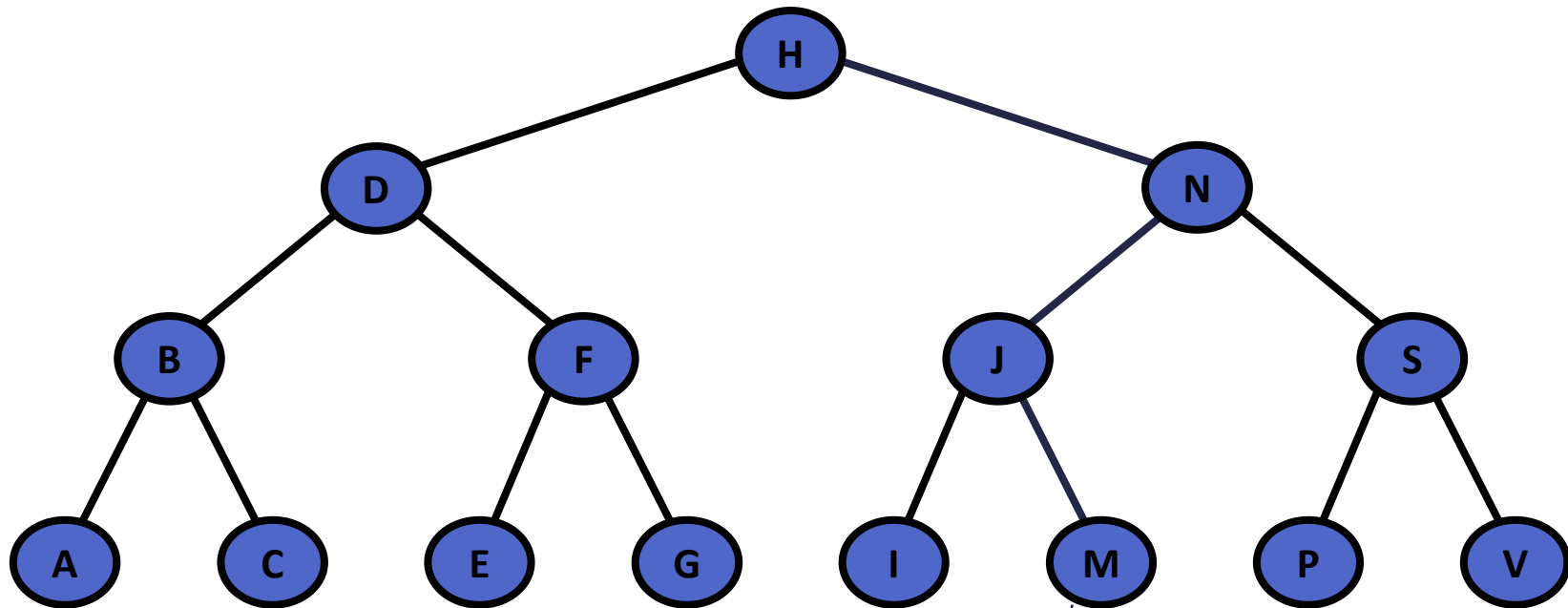


L?

Problémy vyhledávání

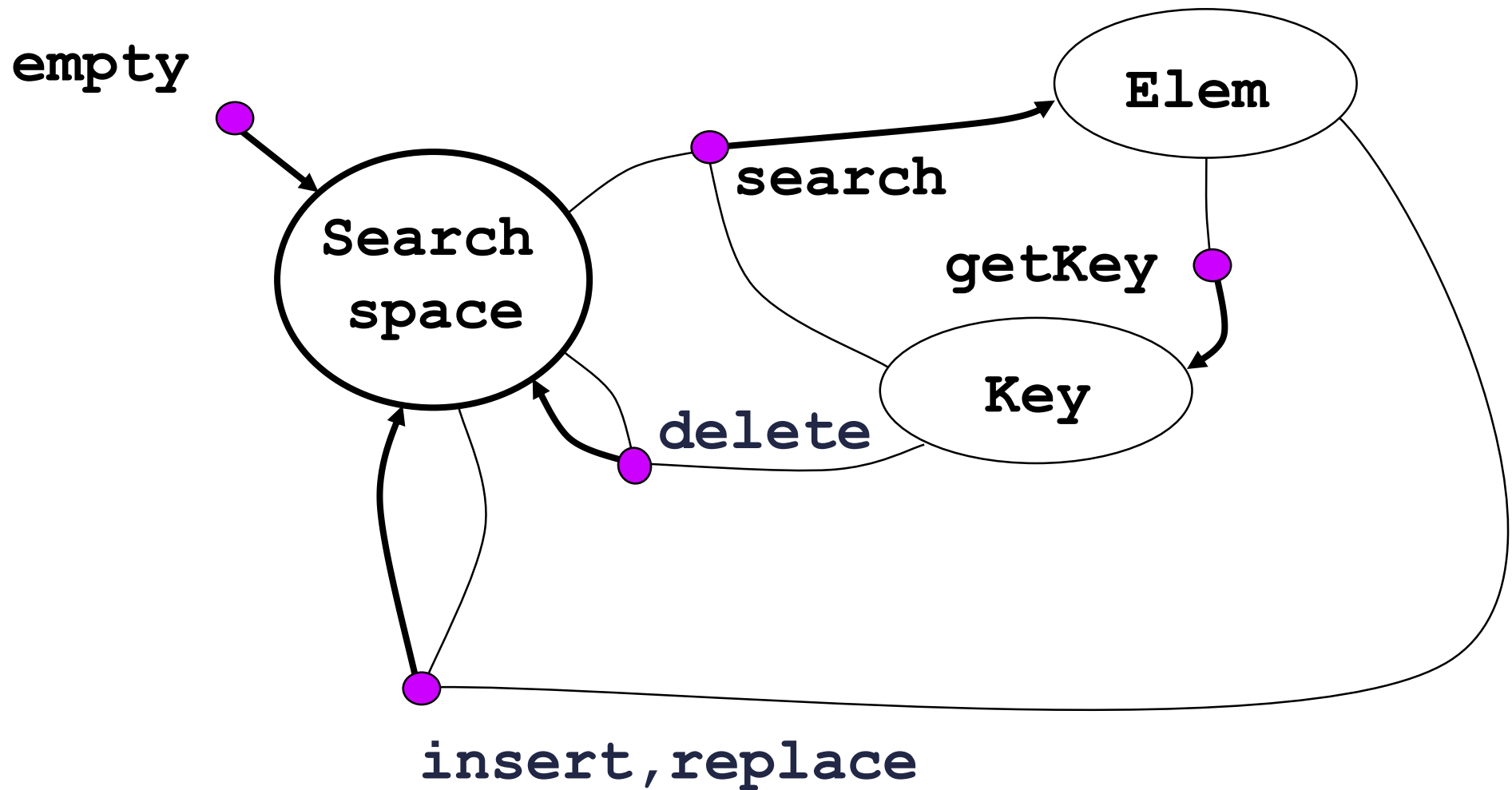
Vstup: množina n klíčů a hledaný klíč k

Popis problému: Kde je k ?



L nebylo nalezeno

Signatura operací pro vyhledávání

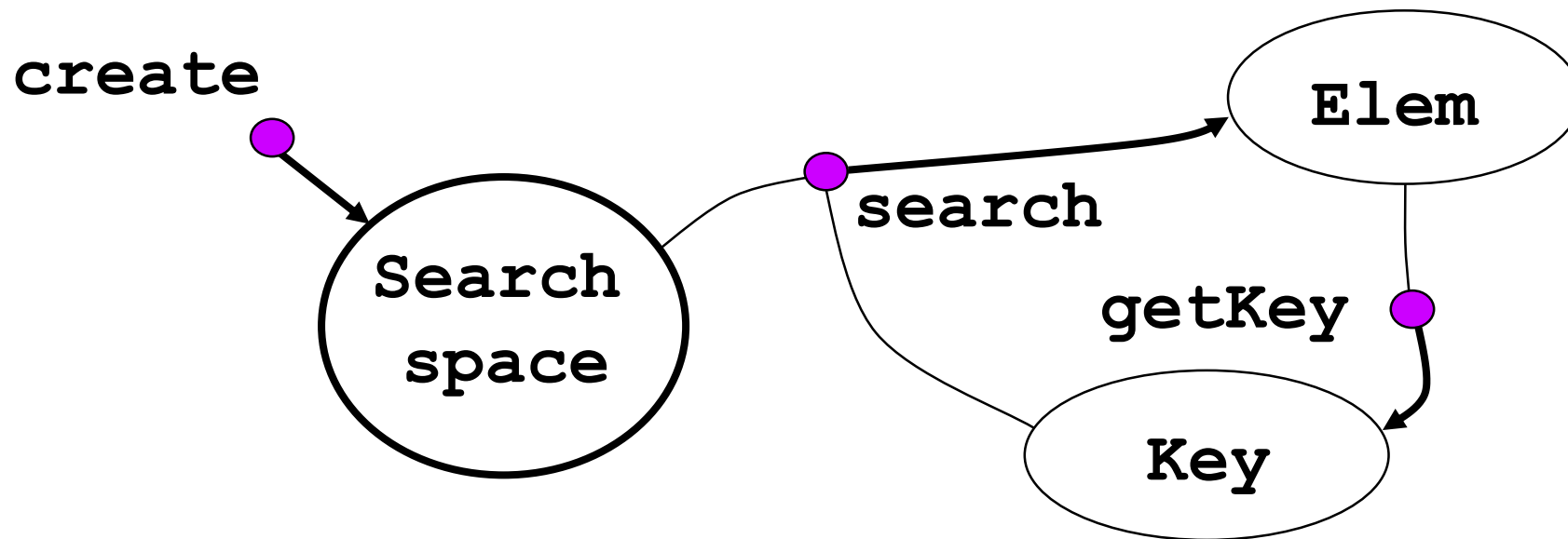


Taxonomie pro vyhledávání

Prohledávaný prostor

- **Statický**
 - prohledávaný prostor je fixní
 - > jednodušší implementace
 - > změna znamená novou verzi
 - > příklad: telefonní seznam, tištěný adresář
- **Dynamický**
 - prohledávaný prostor se může měnit v čase
 - > složitější implementace
 - > změny se provádějí pomocí operací:
insert, delete, replace
 - > příklad: tabulka symbolů v kompilátoru

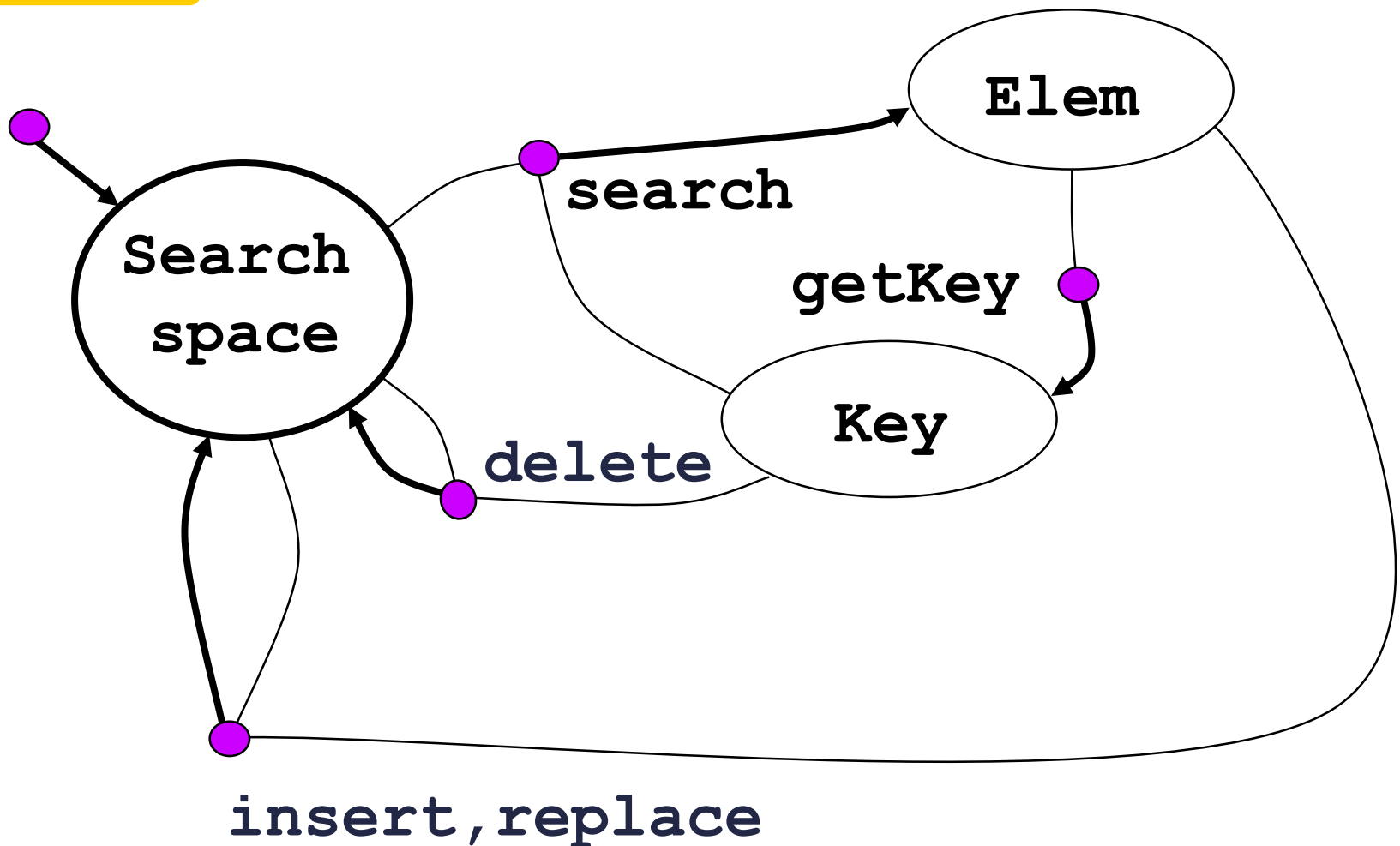
Statický prohledávaný prostor



Dynamický prohledávaný prostor

Dynamic search space

empty



Vyhledávací operace

Symbols:

- k ... klíč (key)
- e ... element s klíčem k (element with key k)
- s ... prohledávaná sada (data set)

Operace (informativní seznam):

- Selektory:

- $\text{search}(k, s)$
 - $\text{min}(s), \text{max}(s)$
 - $\text{pred}(e, s), \text{succ}(e, s)$
- } extenze

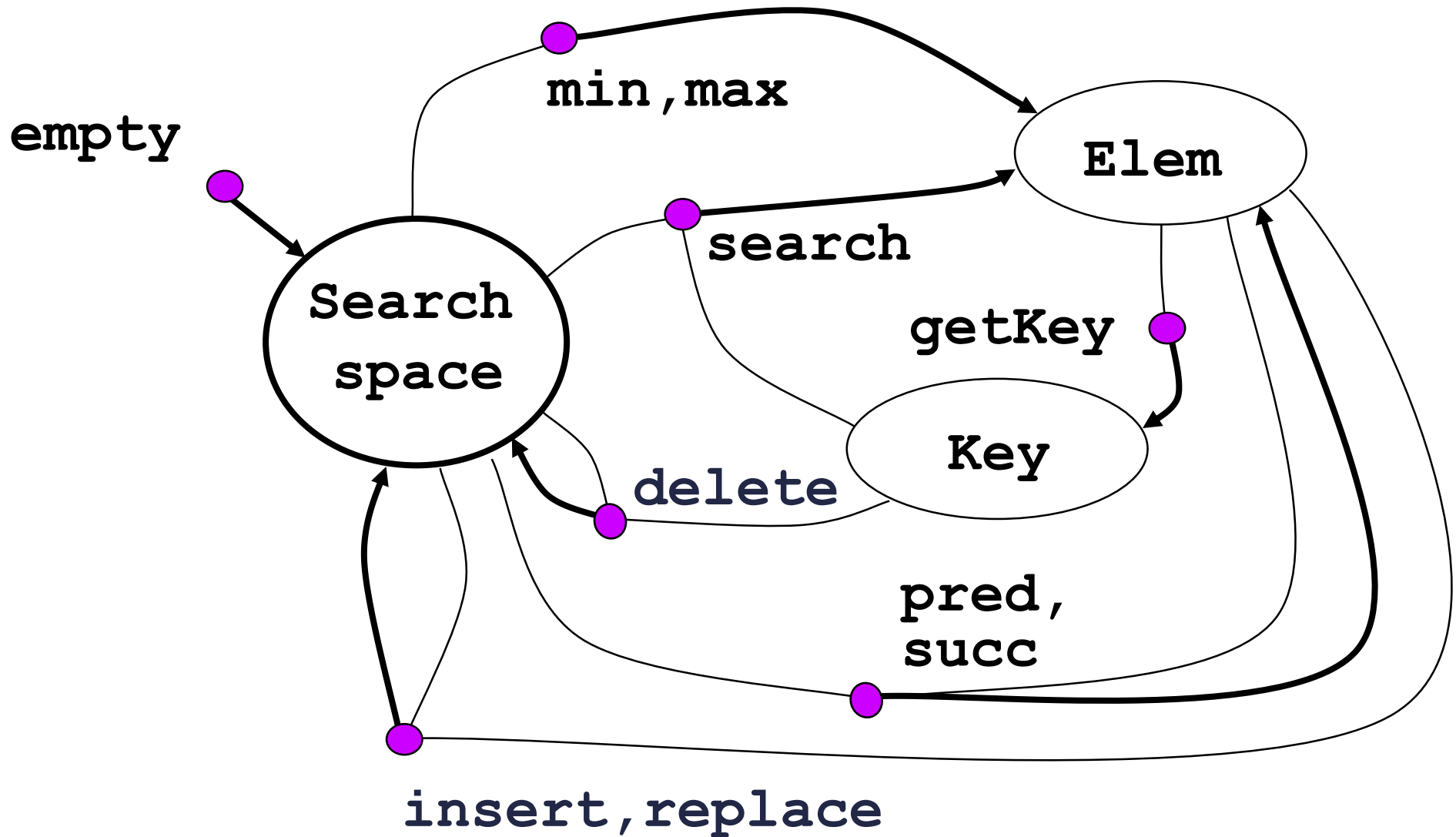
klíč
nahrazovaného
elementu je
součástí nového
elementu e



- Modifikátory:

- $\text{insert}(e, s), \text{delete}(k, s), \text{replace}(e, s)$

Rozšířené vyhledávání



Jiná klasifikace

Adresní vyhledávání

- Adresní vyhledávání - založeno na digitálních vlastnostech klíčů
 - výpočet pozice z klíče $\text{pos} = f(k)$
 - přímý přístup (direct access), hašování
 - pole, tabulka, ...
 - přímý přístup $\Rightarrow$ rychlé ... $O(1)$

Asociativní vyhledávání

- Asociativní vyhledávání - založeno na porovnání elementů
 - elementy jsou umístěny podle vztahu k ostatním
 - sekvenční hledání, binární hledání, vyhledávací stromy
 - vyžaduje hledání $\Rightarrow$ pomalejší ... $O(\log n)$ až $O(n)$

Ještě jiné klasifikace

Interní nebo externí

- Interní nebo externí
 - Interní – v paměti
 - Externí – na disku (příp. pásce)

Dimenze klíčů

- Dimenze klíčů
 - Jedno-dimenzionální - k
 - Multi-dimenzionální - $[x,y,z]$

Měření kvality (Quality measures)

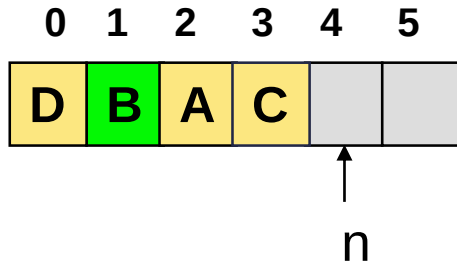
Prostor pro data

- $P(n)$ = paměťová složitost (memory complexity)

Časová náročnost

- $Q(n)$ = časová složitost **search**, **query**
- $I(n)$ = časová složitost **insert**
- $D(n)$ = časová složitost **delete**

Sekvenční vyhledávání v neseřazeném poli



- **Neseřazené pole**
- sekvenční hledání
- insert
- delete
- min, max

$P(n) = O(n)$

$Q(n) = O(n)$

$I(n) = O(1)$

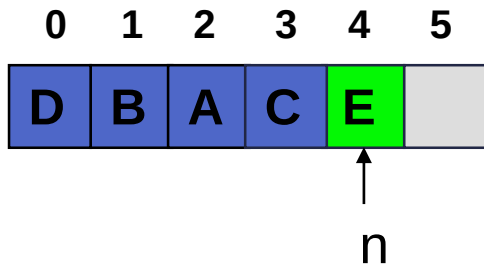
$D(n) = O(n)$

in $O(n)$



```
Elem seqSearch( key k, Elem a[], int n ) {  
    int i = 0;  
    while( ( i < n ) && ( a[i].key != k ) ) i++;  
    if( i < n ) return a[i];  
    else return NODE_NOT_FOUND;  
}
```

Vylepšené vyhledávání v neseřazeném poli



Neseřazené pole se zarážkou (sentinel)

Sekvenční hledání je stále $Q(n) = O(n)$

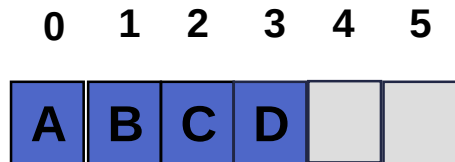


`search("E", a, n)`

```
Elem seqSearchWithSentinel( key k, Elem a[], int n ) {  
    int i = 0;  
    a[n] = createArrayElement(k); // add sentinel  
    while( a[i].key != k ) // save one test per step  
        i++;  
    if( i < n ) return a[i];  
    else return NODE_NOT_FOUND;  
}
```



Binární hledání v seřazeném poli



n

Seřazené pole

Binární hledání

insert

delete

min, max

$P(n) = O(n)$

$Q(n) = O(\log(n))$

$I(n) = O(n)$

$D(n) = O(n)$

$O(1)$



```
Elem binarySearch( key k, Elem sortedArray[], int n ) {  
    int pos = bs( k, sortedArray, 0, n - 1 );  
  
    if( pos >= 0 ) return sortedArray[pos];  
    else  
        return NODE_NOT_FOUND;  
        // bs can return -(pos+1), i.e.  
        // position to insert the node with key k  
}
```

Binární hledání – rekurze/iterace

// Recursive version

```
int bs( key k, Elem a[], int first, int last ) {  
    if( first > last ) return -(first + 1); // not found  
    int mid = ( first + last ) / 2;  
    if( k < a[mid].key ) return bs( k, a, first, mid - 1);  
    if( k > a[mid].key ) return bs( k, a, mid + 1, last );  
    return mid; // found!  
}
```

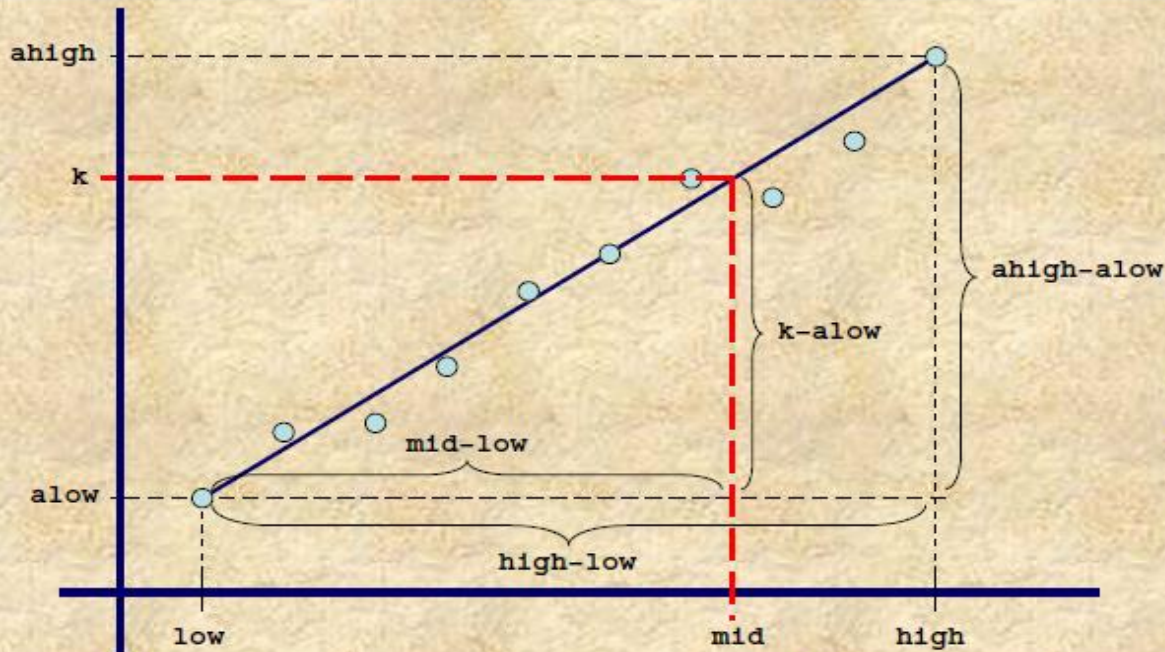
// Iterative version

```
int bs(key k, Elem a[], int first, int last ) {  
    while (first <= last) {  
        int mid = (first + last) / 2; // mid point  
        if (k > a[mid].key) first = mid + 1;  
        else if (key < a[mid].key) last = mid - 1;  
        else return mid; // found  
    } return -(first + 1); // failed to find key  
}
```


Interpolační vyhledávání

Jak hledáme ve slovníku nebo v tlř seznamu? Určitě ne půlením ...

Předpokládejme **rovnoměrné rozložení (číselných) klíčů**, umístění klíče odhadujeme podle jeho hodnoty interpolací:



$$\frac{(k - \text{alow})}{(\text{ahigh} - \text{alow})} = \frac{(\text{mid} - \text{low})}{(\text{high} - \text{low})}$$



$$\text{mid} = \text{low} + (k - \text{alow}) * (\text{high} - \text{low}) / (\text{ahigh} - \text{alow})$$

Porovnání binární/interpolační hledání

Binární hledání - $O(\log n)$

```
mid = ( first + last ) / 2;
```

Interpolační hledání - $O(\log \log n)$

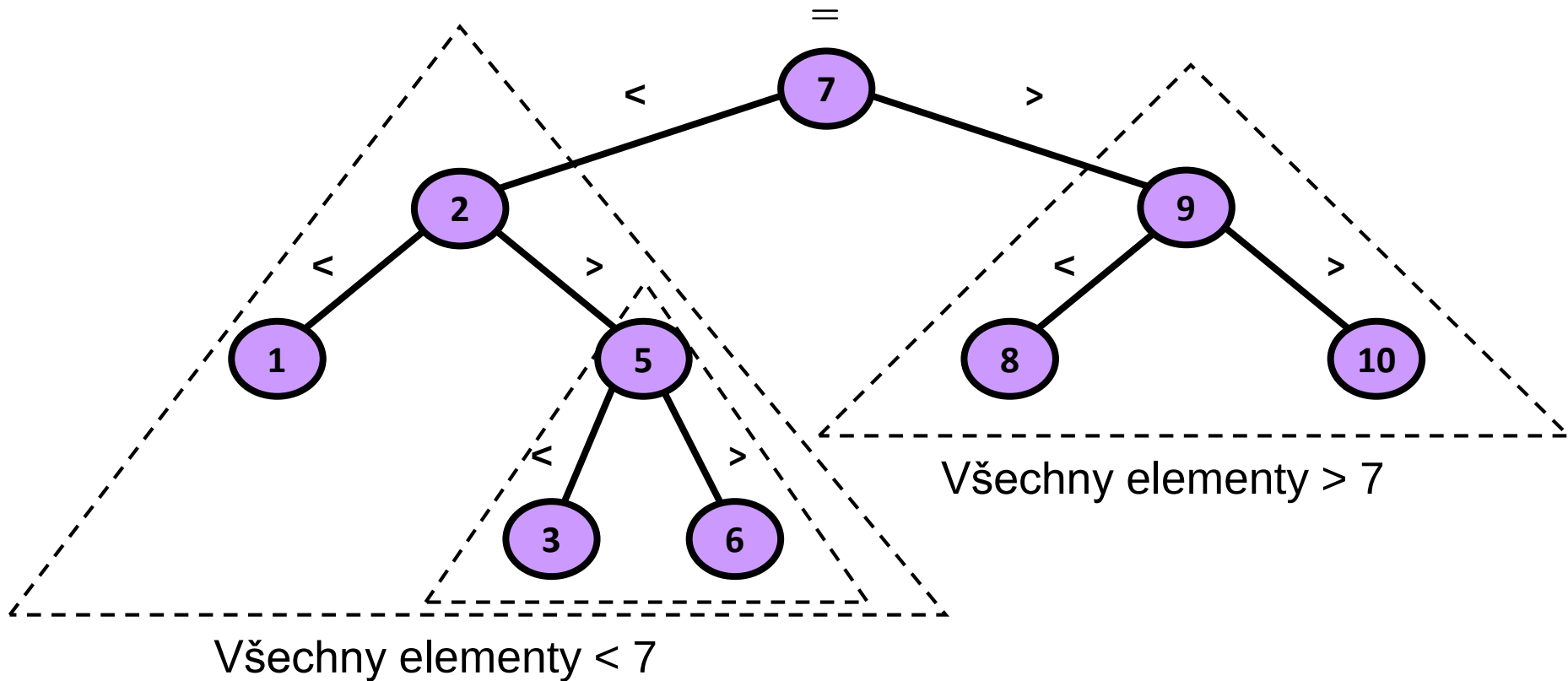
$$\text{pos} = \text{first} + \frac{(\text{last} - \text{first})}{\text{a}[\text{last}] - \text{a}[\text{first}]} (\text{x} - \text{a}[\text{first}])$$

```
Elem searchInterpol( Elem a[], int low, int high, Key k) {  
    if (low > high) return NULL;  
    int mid = low + (k-a[low].key)*(high-low)/(a[high].key-a[low].key);  
    if (a[mid].key == k) return a[mid];  
    if (a[mid].key < k)  
        return searchInterpol(Elem[] a, mid+1, high, k );  
    else return searchInterpol(Elem[] a, low, mid-1, k );  
}
```

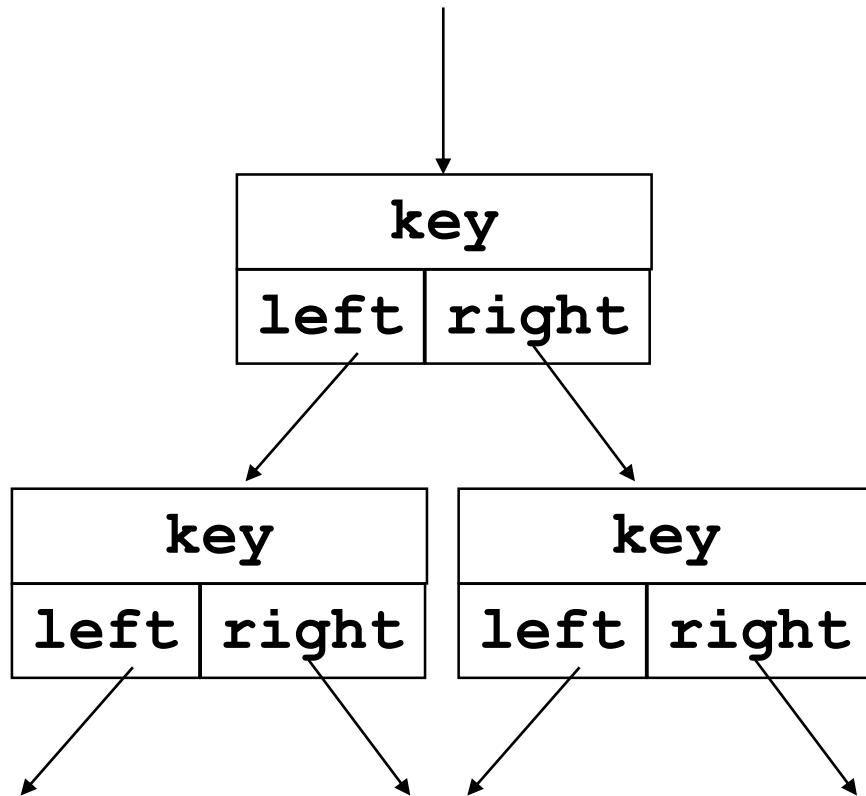
Kořenový binární vyhledávací strom (BVS)

- *Kořenový binární strom*
 - uzel má 0, (1), 2 následníky (nemusí být pravidelný)
- **Binární vyhledávací strom (BVS)**
 - uspořádaný kořenový binární strom
 - u je kořen
 - pro všechny uzly u_L z levého podstromu platí:
 $\text{klíč}(u_L) < \text{klíč}(u)$
 - pro všechny uzly u_R z pravého podstromu platí:
 $\text{klíč}(u_R) > \text{klíč}(u)$

Binární vyhledávací strom (BVS)



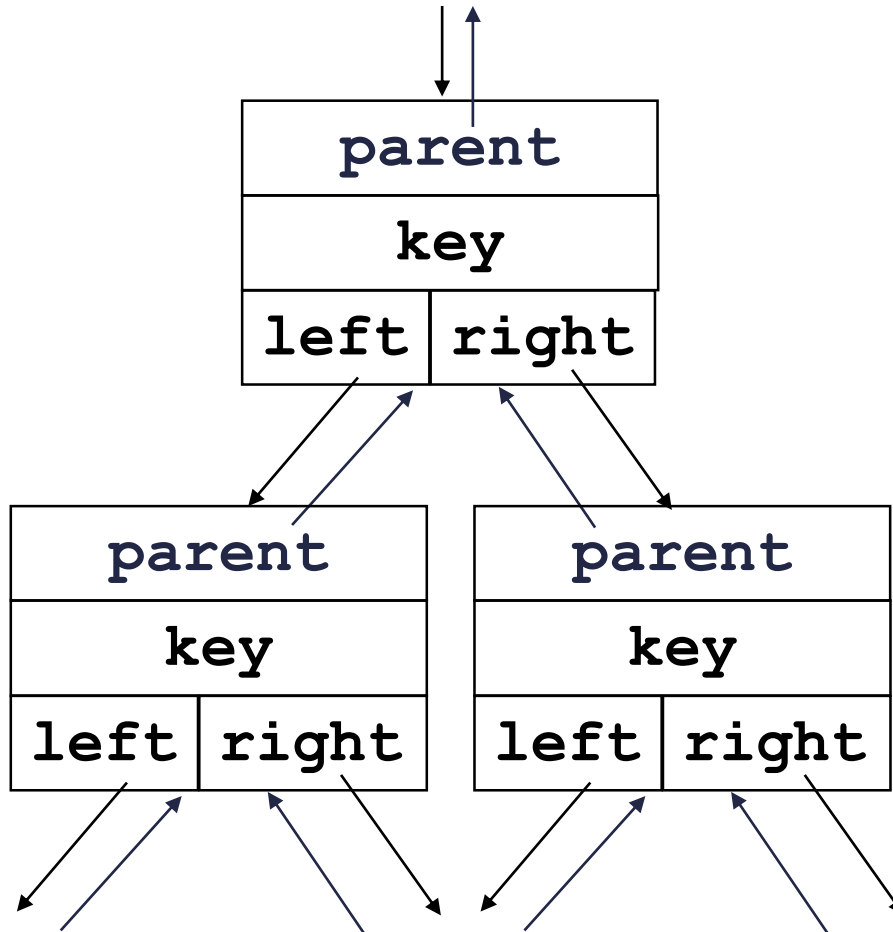
Reprezentace uzlů stromu



Výhodné pro:

- search
- min, max

Reprezentace uzlů stromu II.



Vhodné pro:

- search
- min, max
- pred, succ

Reprezentace uzlů stromu

```
/* bez odkazu na rodiče */
```

```
struct Node {  
    Node *left;  
    Node *right;  
    Elem key;  
};
```

```
Node *root = NULL;
```

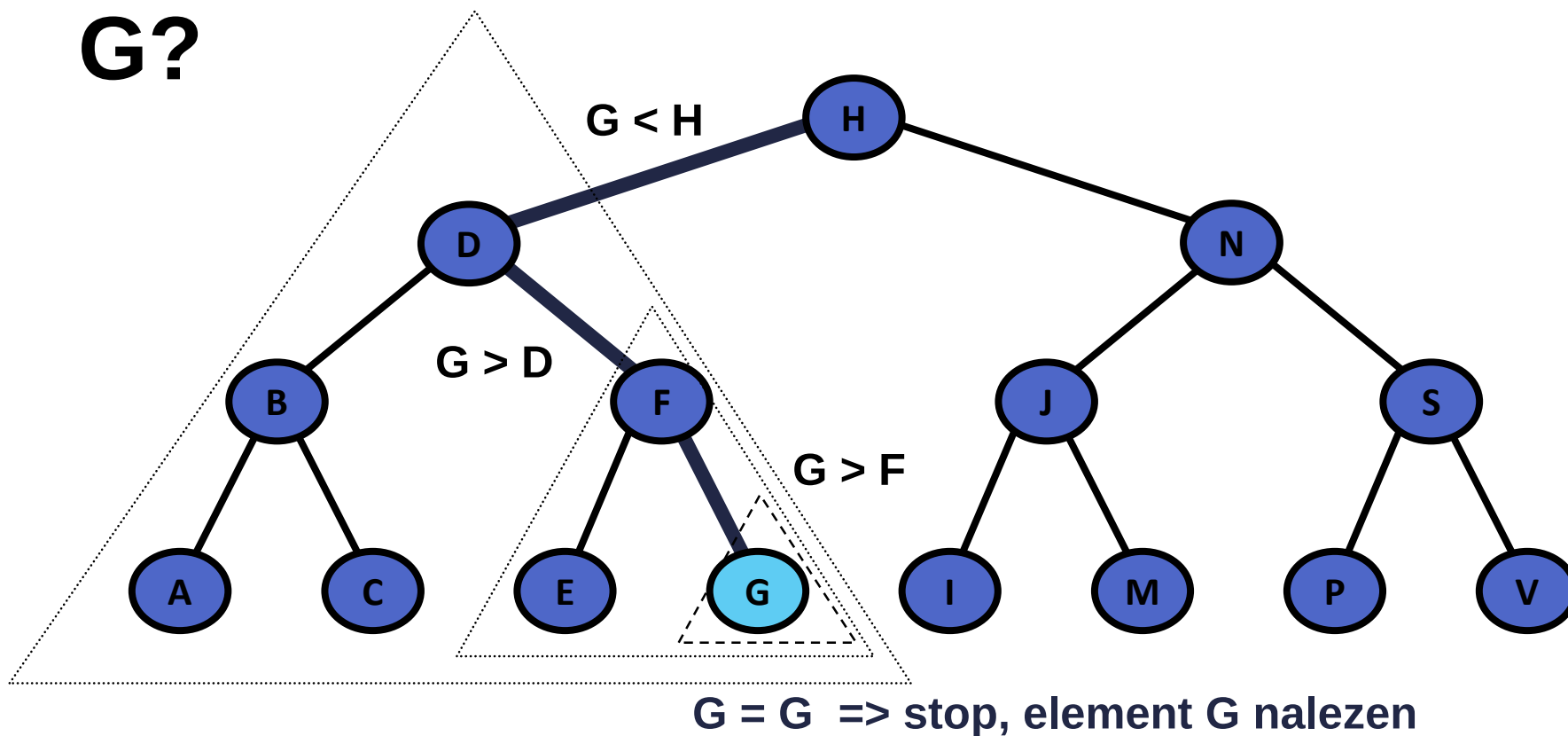
```
/* s odkazem na rodiče */
```

```
struct Node {  
    Node *left;  
    Node *right;  
    Node *parent;  
    Elem key;  
};
```

```
Node *root = NULL;
```

Prohledávání BVS

G?

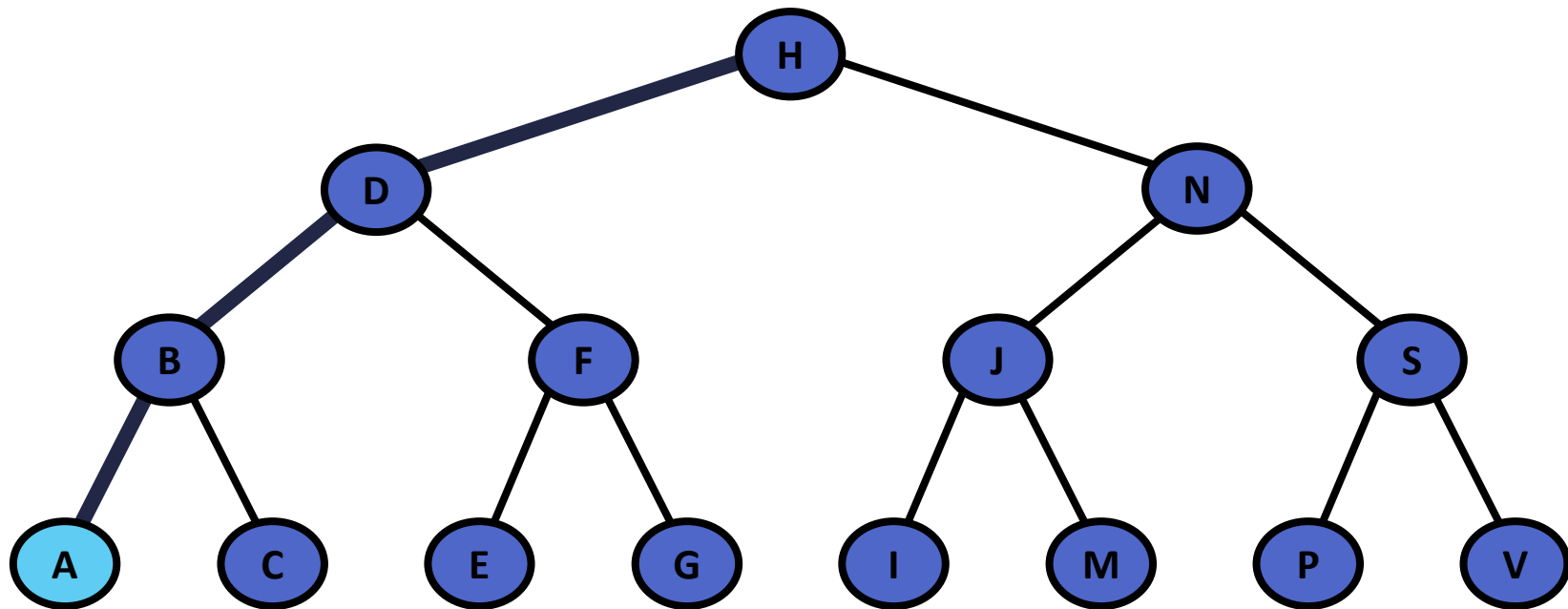


Prohledávání BVS – rekurzivně/iterativně

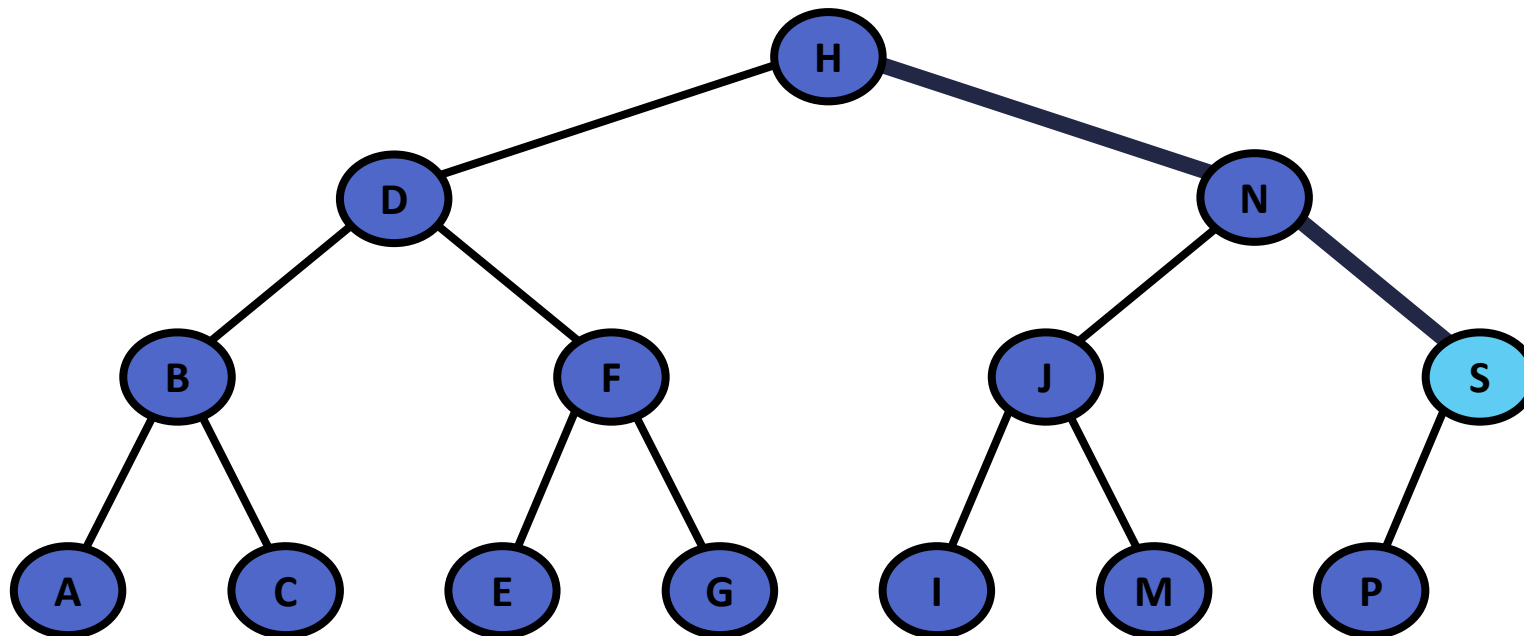
```
//Recursive version
Node *treeSearch( Node *x, key k )
{
    if(( x == NULL ) || ( k == x->key ))
        return x;
    if( k < x->key )
        return treeSearch( x->left, k );
    else
        return treeSearch( x->right, k );
}
```

```
//Iterative version
Node *treeSearch( Node *x, key k )
{
    while (( x != NULL ) && ( k != x->key ))
        if( k < x->key ) x = x->left;
        else x = x->right;
    return x;
};
```

Minimum pro BVS



Maximum pro BVS



Minimum a mazimum pro BVS – iterativně

```
Node *treeMinimum( Node *x )
{
    if( x == NULL ) return NULL;
    while( x->left != NULL )
    {
        x = x->left;
    }
    return x;
}
```

```
Node *treeMaximum( Node *x )
{
    if( x == NULL ) return NULL;
    while( x->right != NULL )
    {
        x = x->right;
    }
    return x;
}
```

Následník v BVS

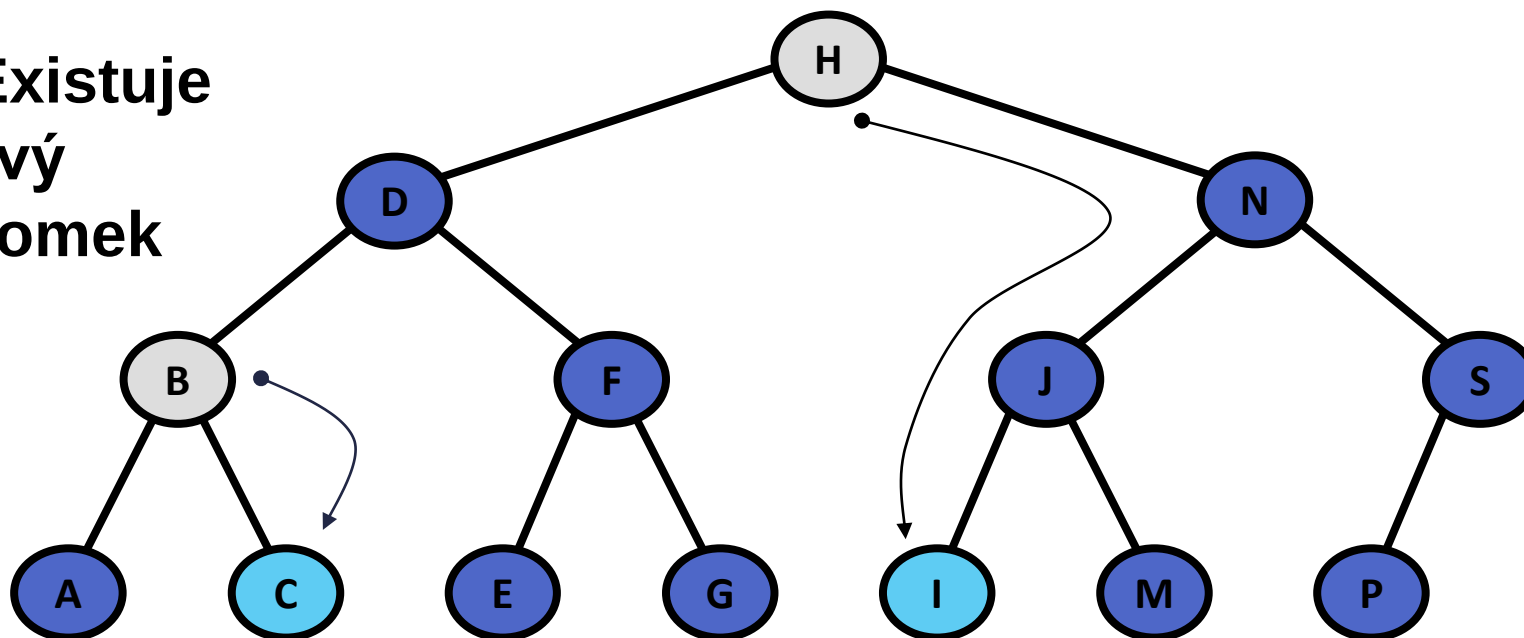
podle uspořádání (in-order tree walk)

- Jsou dvě možnosti:
 1. **Existuje pravý potomek**
 2. Neexistuje pravý potomek

Následník v BVS

podle uspořádání (in-order tree walk)

1. Existuje
pravý
potomek

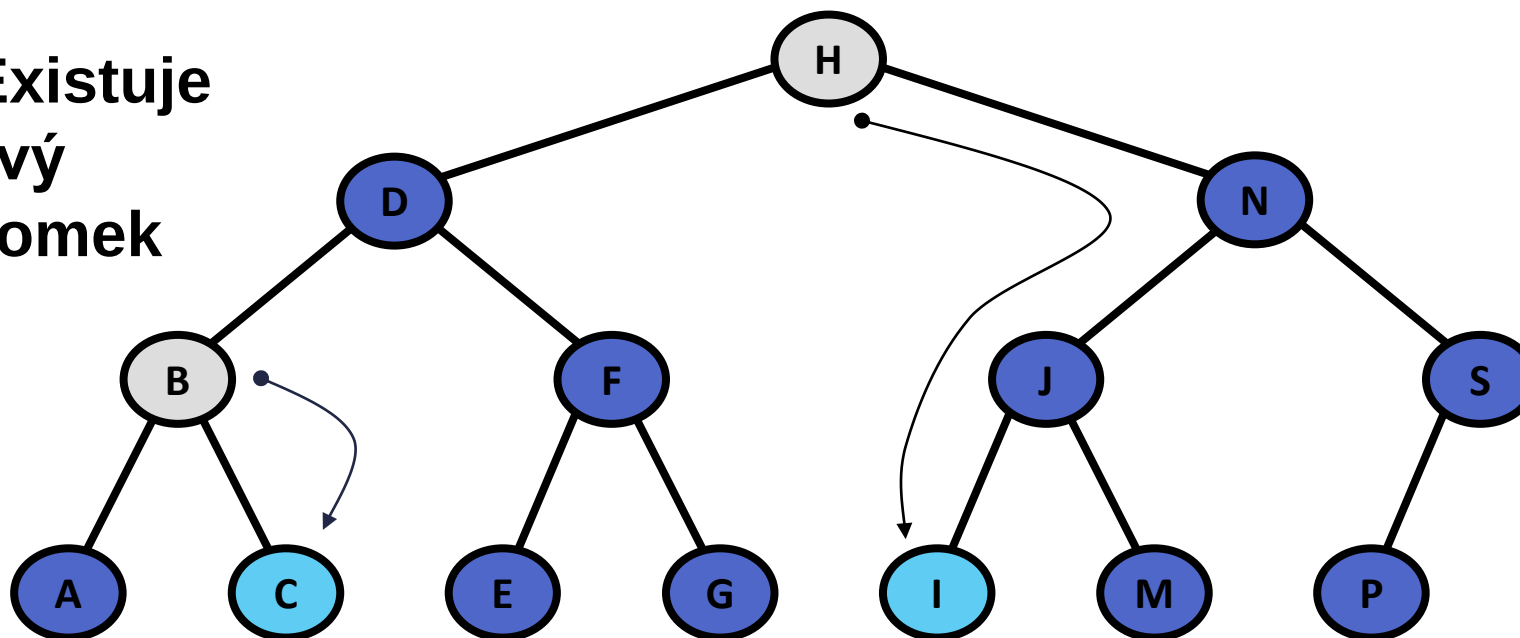


$\text{succ}(B) \rightarrow C$
 $\text{succ}(H) \rightarrow I$ **Jak?**

Následník v BVS

podle uspořádání (in-order tree walk)

1. Existuje
pravý
potomek



$\text{succ}(B) \rightarrow C$

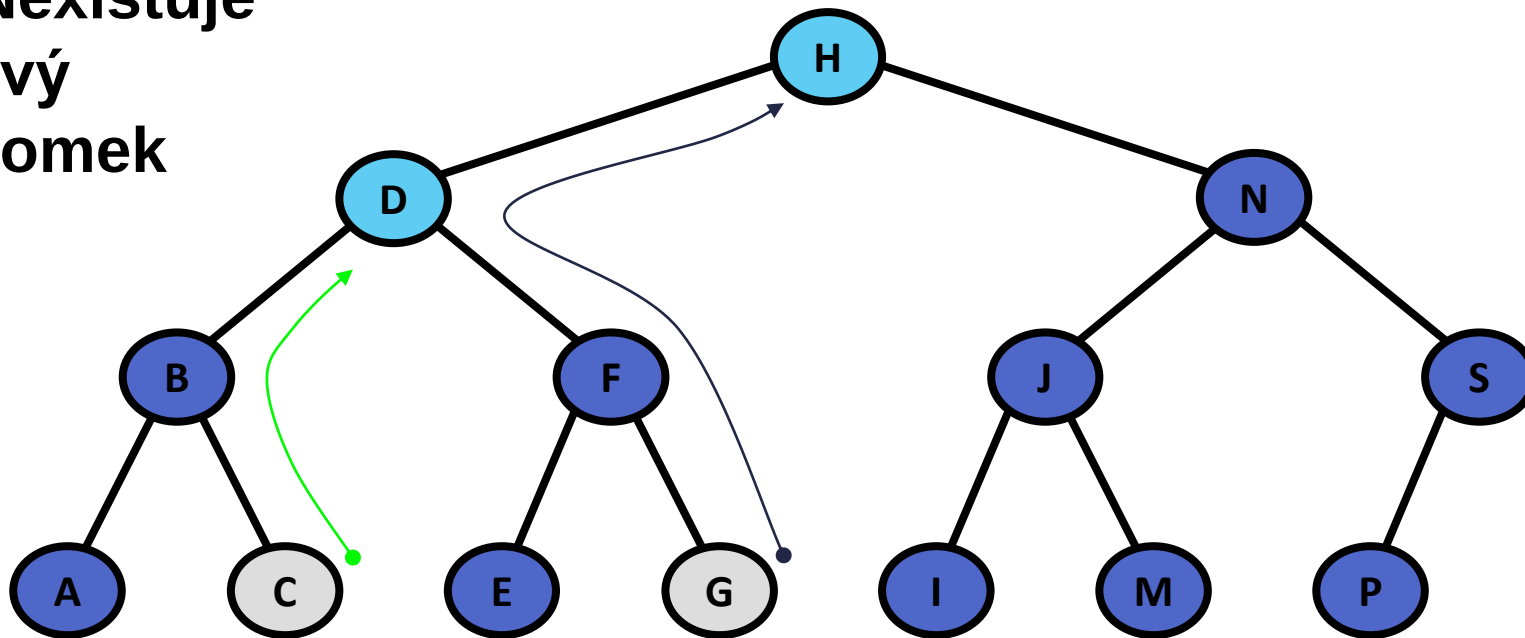
$\text{succ}(H) \rightarrow I$

Najdi *minimum* v pravé větvi
 $= \min(x \rightarrow \text{right})$

Následník v BVS

podle uspořádání (in-order tree walk)

1. Nexistuje
pravý
potomek



succ (C) -> D

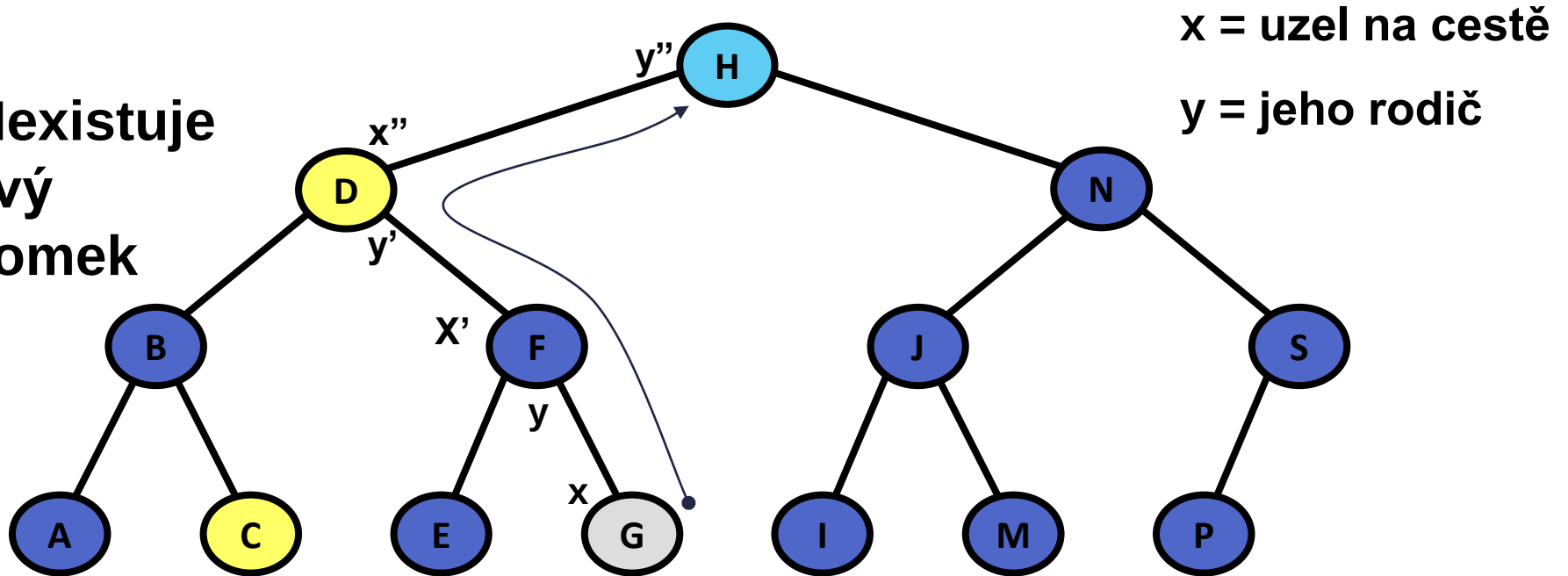
succ (G) -> H

} Jak?

Následník v BVS

podle uspořádání (in-order tree walk)

1. Nexistuje
pravý
potomek



$\text{succ}(G) \rightarrow H$

Najdi *minimálního* rodiče vpravo
(minimální rodič na cestě do kořene)

Následník v BVS

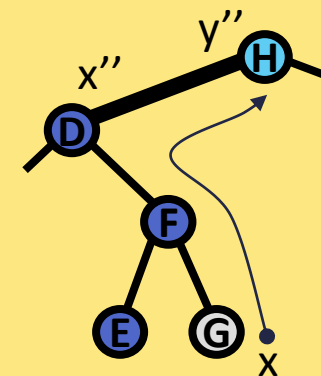
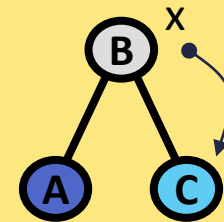
podle uspořádání (in-order tree walk)

x = uzel na cestě, y = jeho rodič

```
Node *treeSuccessor( Node *x )
{ Node *y;
  if( x == NULL ) return NULL;

  if( x->right != NULL ) // 1. right son exists
    return treeMinimum( x->right );

  y = x->parent;          // 2. right son is NULL
  while( (y != NULL) && (x == y->right))
  {
    x = y;
    y = x->parent;
  }
  return y;               // first parent x is left from
};
```



Předchůdce v BVS

podle uspořádání (in-order tree walk)

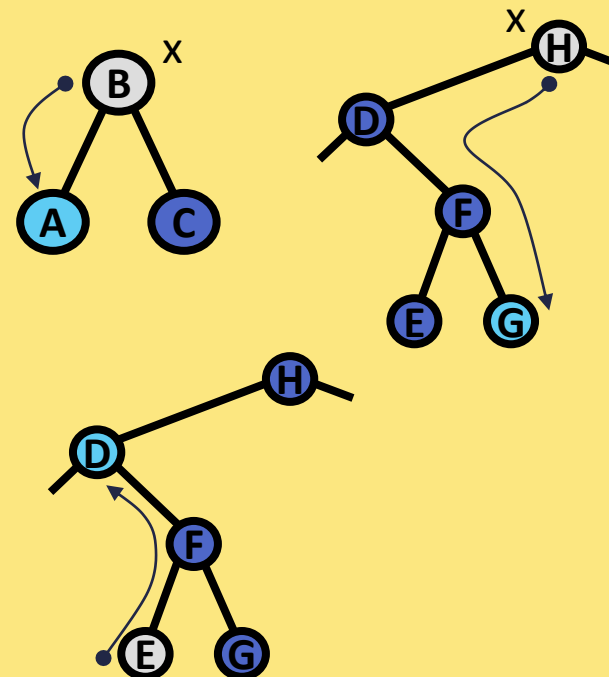
x = uzel na cestě, y = jeho rodič

```
Node *treePredecessor( Node *x )
{ Node *y;

  if( x == NULL ) return NULL;

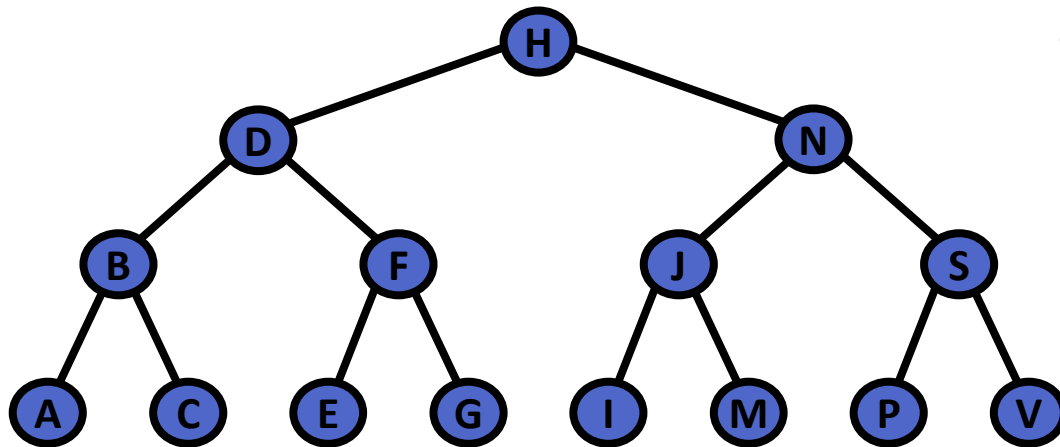
  if( x->left != NULL )
    return treeMaximum( x->left );

  y = x->parent;
  while( (y != NULL) && (x == y->left))
  {
    x = y;
    y = x->parent;
  }
  return y;
}
```



Operační složitost

Dynamické operace **search**, **max**, **min**, **succ**, **pred** mají operační složitost $O(h)$, kde h je hloubka vyhledávacího stromu.

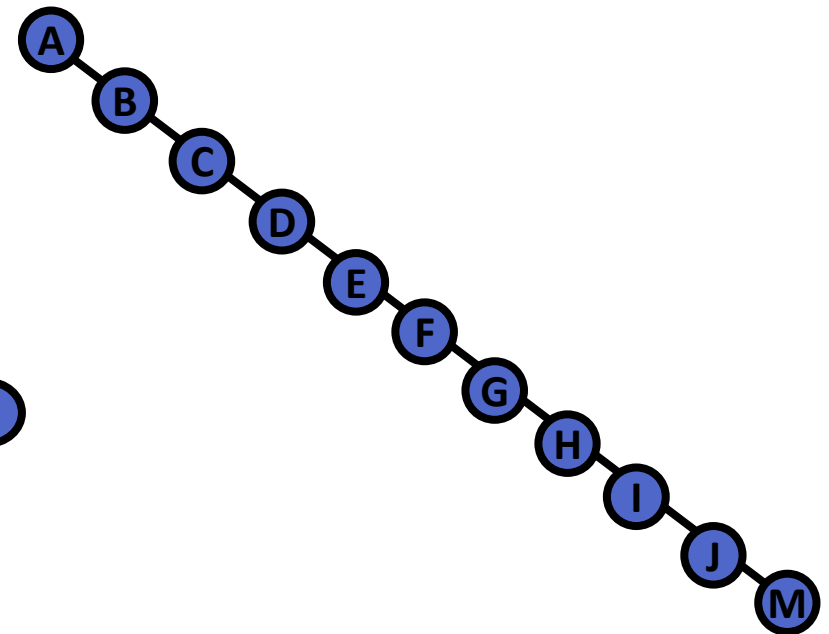


$$h = \log_2(n)$$

$$\Rightarrow O(\log(n))$$



$\Rightarrow$ strom je třeba vyvážit!!!

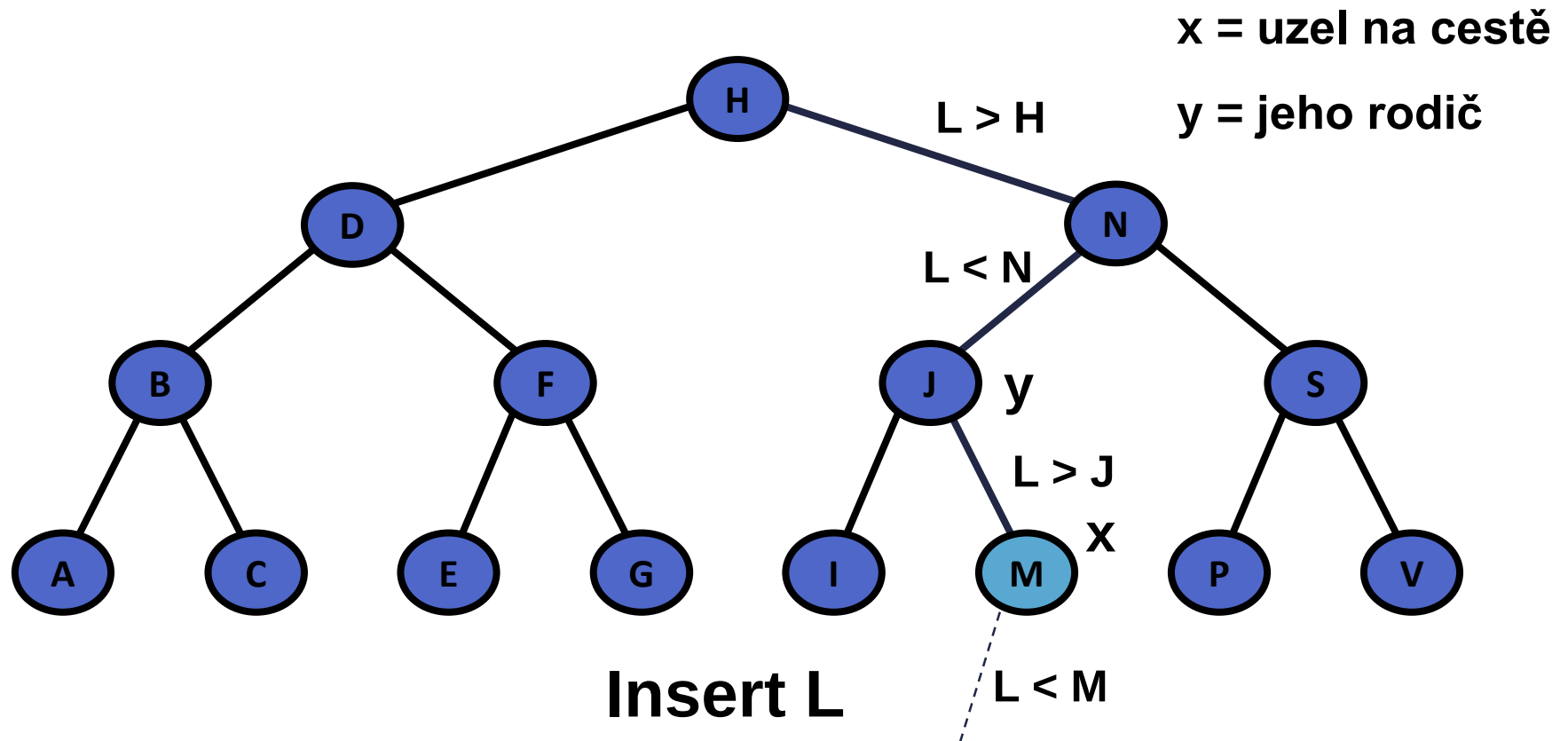


$$h = n$$

$$\Rightarrow O(n) !!!$$



Insert (vložení prvku)



1. Najdi list, který bude rodičem ... M
2. Připoj nový element jako nový list ... M.left

Insert (vložení prvku)

x = uzel na cestě, y = jeho rodič

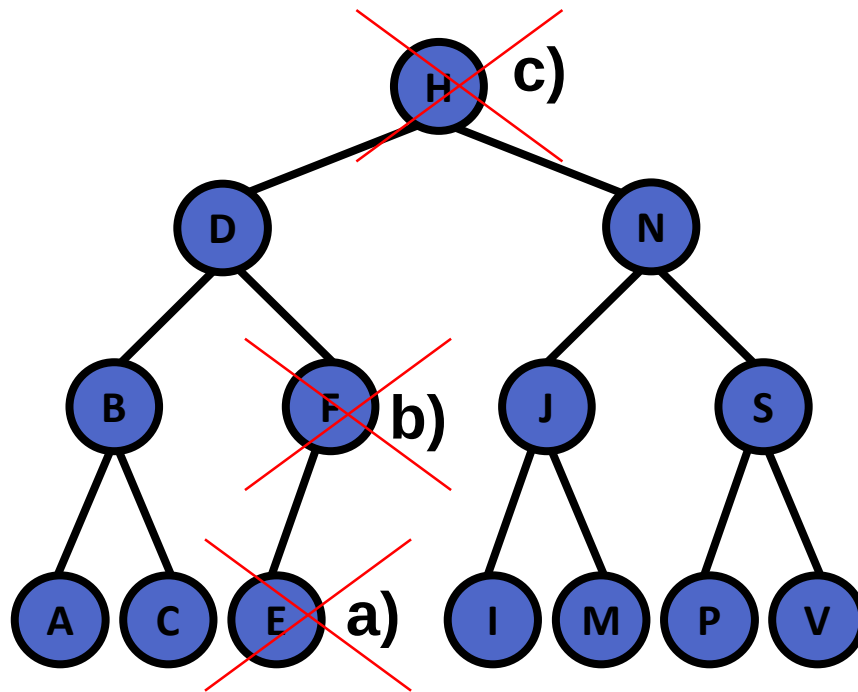
```
void treeInsert( Tree *t, Node *e )
{
    x = t->root; y = NULL;  // set x to tree root

    if( x == NULL )
        t->root = e;        // tree was empty
    else {
        while( x != NULL ) { // find the parent leaf
            y = x;
            if( e->key < x->key ) x = x->left;
            else                  x = x->right;
        }
        if( e->key < y->key ) y->left = e; // add e to parent y
        else                  y->right = e;
    }
}
```

Jednoduchá verze – bez „update“ pro stejné klíče.

Operační složitost: najdi list + vložení = $O(\log_2 n) + O(1) = O(\log_2 n)$

Delete (odstranění prvku)

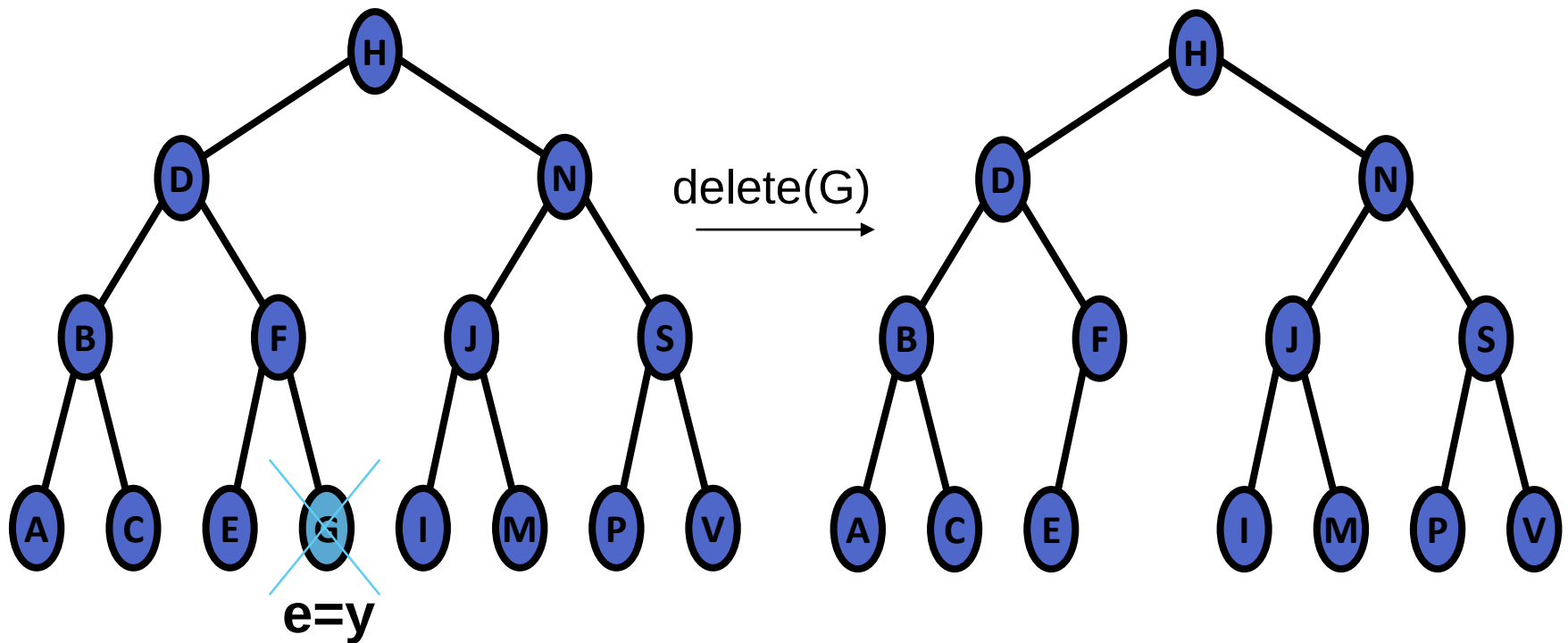


Delete – 3 případy

- a) rušíme list, který nemá potomky
- b) rušíme list, který má jednoho potomka
- c) rušíme list, který má dva potomky

Delete (odstranění prvku)

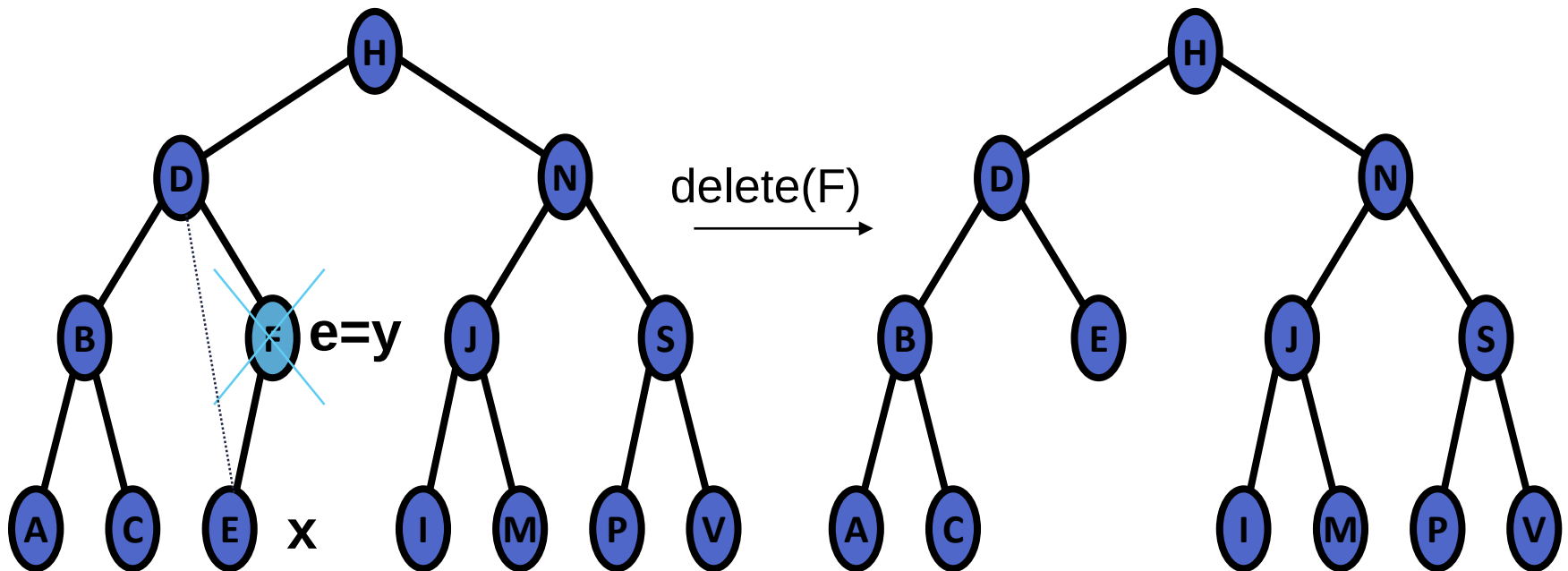
a) smaž list



a) list bez potomků lze jednoduše vypustit

Delete (odstranění prvku)

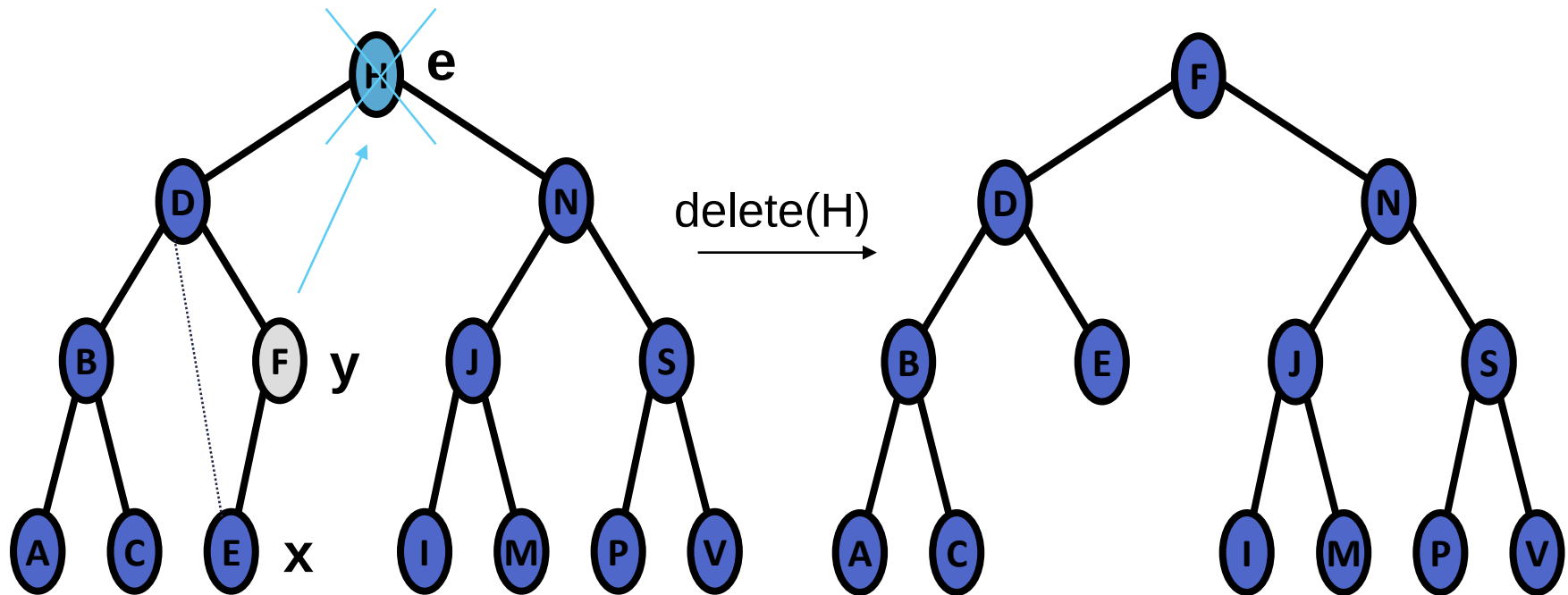
b) vnitřní s potomkem



b) uzel s jedním potomkem - přemostí vymazaný uzel

Delete (odstranění prvku)

c) se 2 potomky



c) uzel se dvěma potomky -> nahrad' uzel jeho předchůdcem (**pred**) a vypust' předchůdce, nebo následníkem (**succ**) a vypust' následníka

Delete (odstranění prvku)

```
Node *treeDelete( Tree *t, Node *e )
```

```
// e...node to logically delete
```

```
// y...node to physically delete
```

```
// x...y's only son
```

```
{ Node *x, *y;
```

```
    1. find node y (e or predecessor of e)
```

```
    2. find x = y's only child or NULL
```

```
    3. link x up with parent of y
```

```
    4. link parent of y down to x
```

```
    5. replace e by in-order predecessor y
```

```
    6. return y (for later use)
```

```
}
```

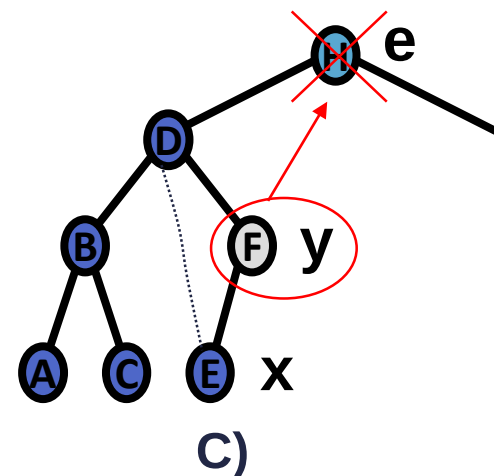
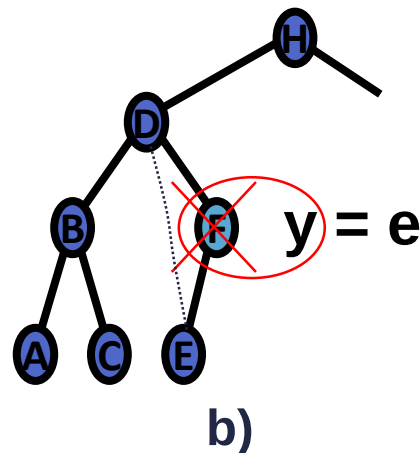
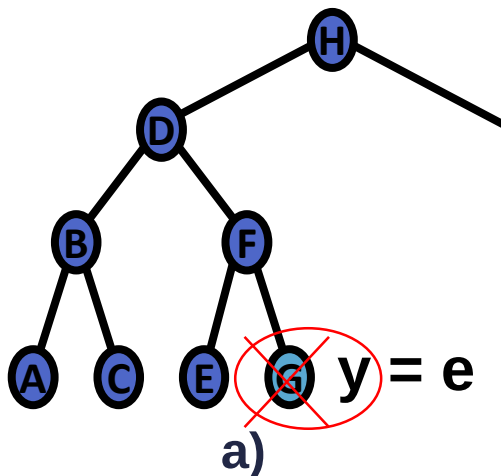
Delete (odstranění prvku) - pokr.

```
Node *treeDelete( Tree *t, Node *e )  
// e...node to logically delete  
// y...node to physically delete  
// x...y's only son
```

```
{ Node *x, *y;
```

```
1.find node y
```

```
if(e->left == NULL OR e->right == NULL) // cases a,b) 0 - 1 child  
    y = e;  
else // case c) 2 children  
    y = TreePredecessor(e);  
    cont...
```



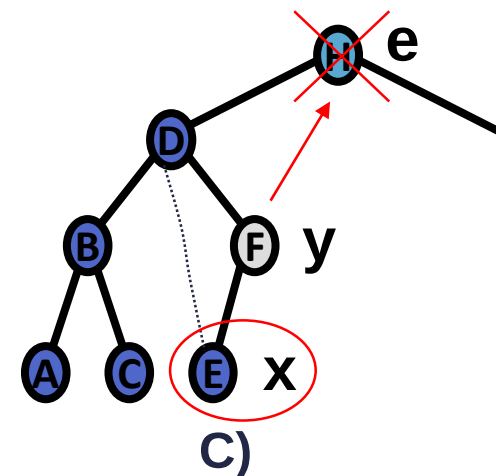
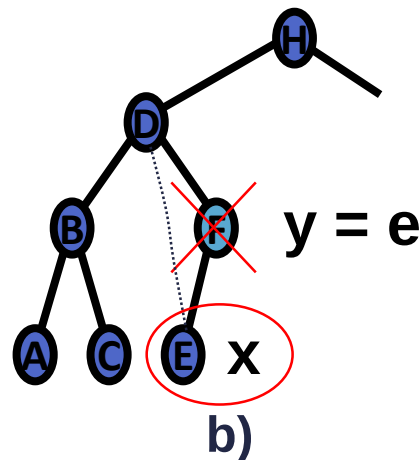
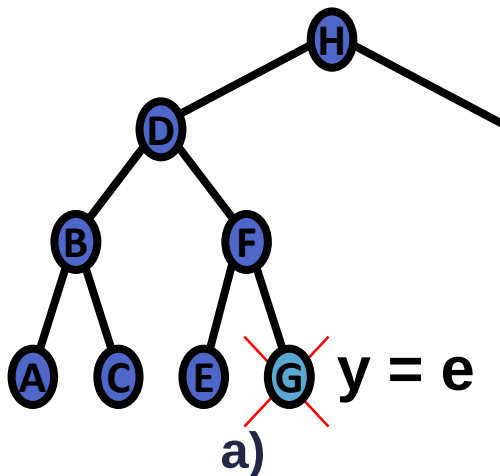
Delete (odstranění prvku)

... cont

2. find $x = y$'s only child or NULL

```
if( y->left != NULL ) // a) NULL, b,c) only child
    x = y->left;
else
    x = y->right;
```

cont...



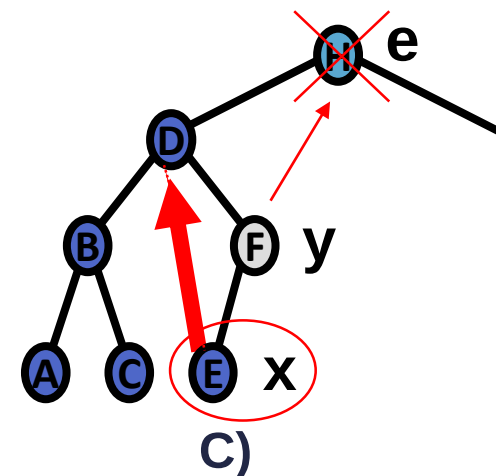
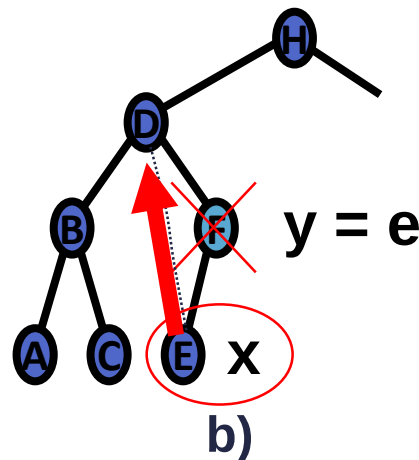
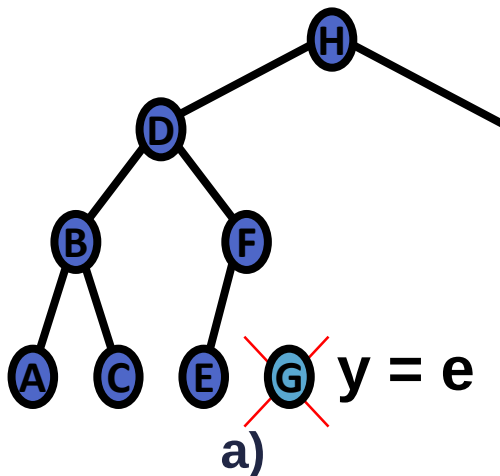
Delete (odstranění prvku)

... cont

3. link x up with its new parent (former parent of y)

```
if( x != NULL ) x->parent = y->parent; // b,c)
```

cont ...

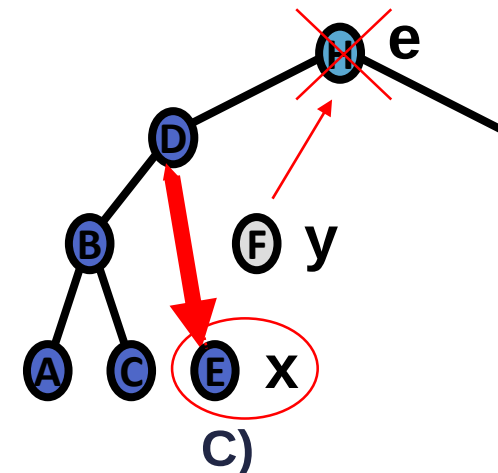
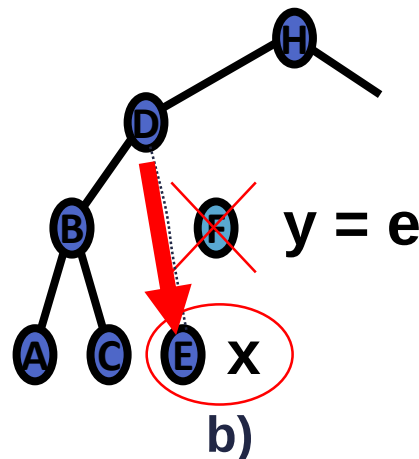
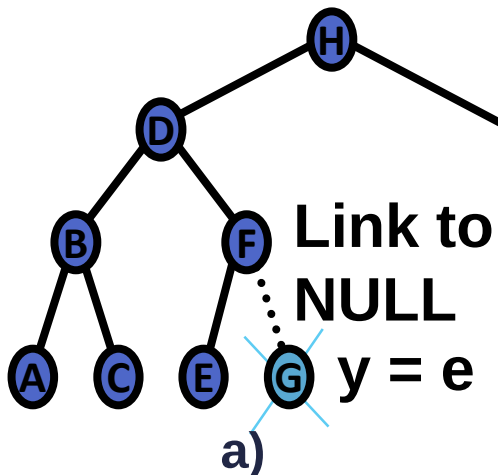
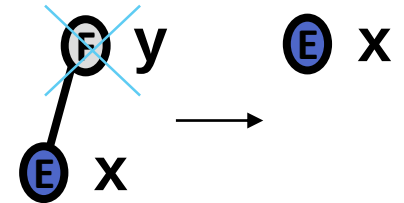


Delete (odstranění prvku)

4. link parent of y down to x

```
if( y->parent == NULL )  
    t->root = x           // y was root  
else if( y == (y->parent)->left )  
    (y->parent)->left = x; // y was left son  
else  
    (y->parent)->right = x; // y was right son
```

cont...



Delete (odstranění prvku)

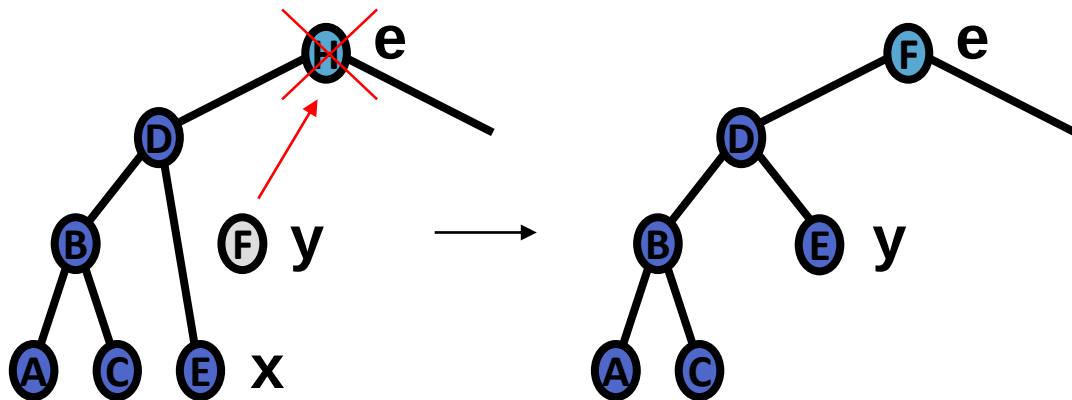
...

5. replace e with in-order predecessor

```
if( y != e ) // replace e with in-order predecessor
{
    e->key = y->key;    // copy the key
    e->data = y->data;  // copy other fields too
}
```

6. return y (for later use)

}



Delete (celkový kód)

```
Node *treeDelete( Tree *t, Node *e )
{ Node *x, *y;
  // e ... node to logically delete
  // y ... node to physically delete, x ... y's only son

  if(e->left == NULL || e->right == NULL)
    y = e;                                     // cases a, b) 0 to 1 child
  else y = TreePredecessor(e);                 // c) 2 children
  if( y->left != NULL )                         // a) NULL, b,c) only child
    x = y->left;
    else x = y->right;
  if( x != NULL ) x->parent = y->parent; // b,c)
  if( y->parent == NULL ) t->root = x          // y-root
  else if( y == (y->parent)->left ) (y->parent)->left = x; // y-L son
    else(y->parent)->right = x;                // y-R son
  if( y != e ) {                               // replace e with in-order predecessor
    e->key = y->key;
    e->dat = y->data; // copy other fields too
  }
  return y;                                     // instead of delete
};
```

Vyvažování stromu (Tree balancing)

Operace nad BVS jsou závislé na míře vyváženosti stromu – pro vyvážený strom dosahují $O(\log_2 n)$, pro nevyvážený $O(n)$.

Vyváženost:

- Silná podmínka – shoda h (Ideální případ)
Pro všechny uzly platí:
počet uzlů vlevo = počet uzlů vpravo
- Slabší podmínka – násobek h
 - **výška** podstromů - AVL strom (jméno pochází z iniciál jeho objevitelů)
 - **výška** + počet potomků - 1-2 strom, ...
 - **váha** podstromů (počty uzlů) - váhově vyvážený strom
 - stejná **černá výška** – Červeno-černý strom

Pomocné operace pro udržení vyvážení stromu – rotace

The End